Table of Contents

Scheduling Algorithm

1. WAP to implement First Come First Serve (FCFS) algorithm.

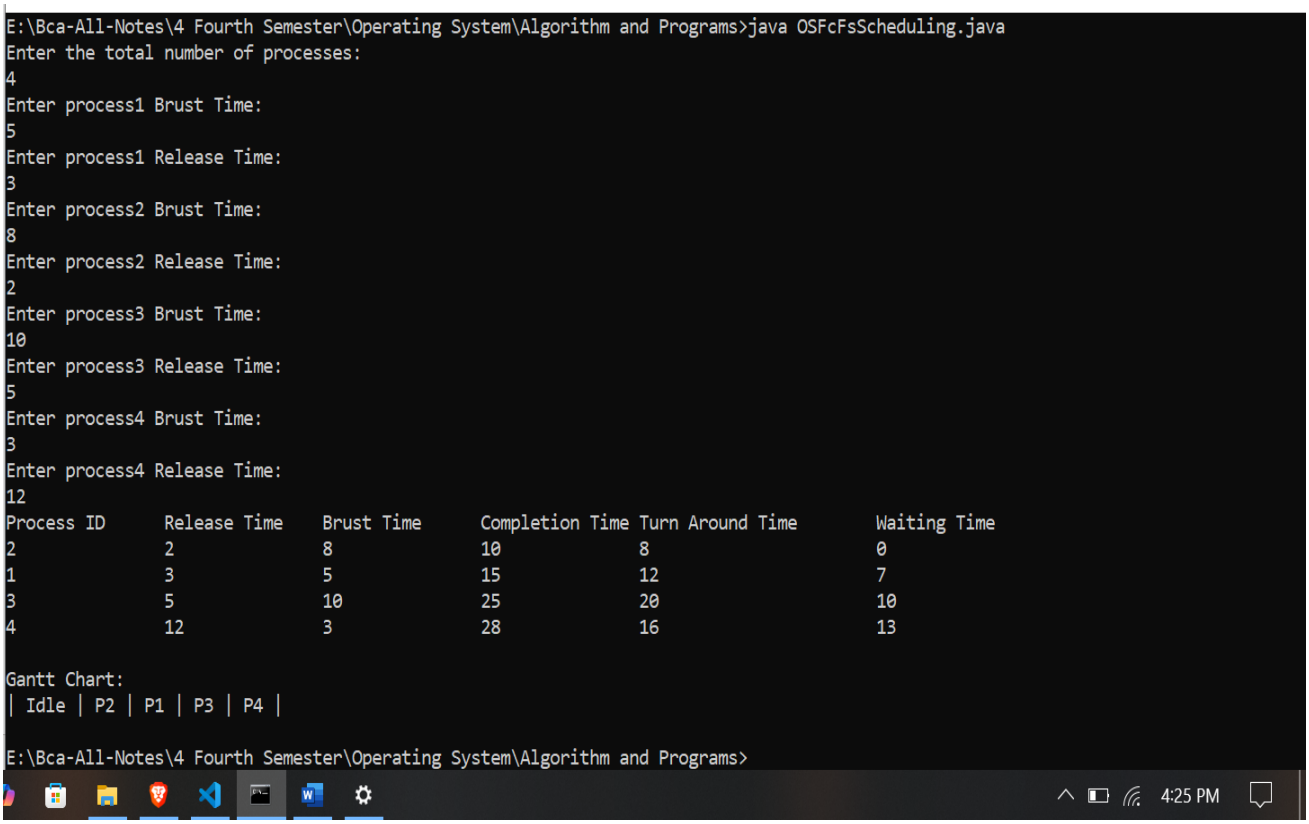
Source Code:

```
import java.util.Scanner;
public class OSFcFsScheduling {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the total number of processes:");
      int PuraProcesses = sc.nextInt();
      int[] processId = new int[PuraProcesses];
      int[] burstTime = new int[PuraProcesses];
      int[] releaseTime = new int[PuraProcesses];
      int[] completionTime = new int[PuraProcesses];
      int[] turnAroundTime = new int[PuraProcesses];
      int[] waitingTime = new int[PuraProcesses];
      for (int p = 0; p < PuraProcesses; p++) {
         System.out.println("Enter process" + (p + 1) + " Brust Time:");
         burstTime[p] = sc.nextInt();
         System.out.println("Enter process" + (p + 1) + " Release Time:");
         releaseTime[p] = sc.nextInt();
         processId[p] = p + 1;
      }
      int asthayi;
      for (int outerLoop = 0; outerLoop < PuraProcesses; outerLoop++) {
         for (int innerLoop = outerLoop + 1; innerLoop < PuraProcesses; innerLoop++) {
            if (releaseTime[outerLoop] > releaseTime[innerLoop]) {
               asthayi = releaseTime[outerLoop];
               releaseTime[outerLoop] = releaseTime[innerLoop];
               releaseTime[innerLoop] = asthayi;
```

```
                asthayi = processId[outerLoop];
                processId[outerLoop] = processId[innerLoop];
                processId[innerLoop] = asthayi;

                asthayi = burstTime[outerLoop];
                burstTime[outerLoop] = burstTime[innerLoop];
                burstTime[innerLoop] = asthayi;
            }
        }
    }
    int currentTime = 0;
    for (int i = 0; i < PuraProcesses; i++) {
        if (currentTime < releaseTime[i]) {
            // CPU is idle until process arrives
            currentTime = releaseTime[i];
        }
        currentTime += burstTime[i];
        completionTime[i] = currentTime;
        turnAroundTime[i] = completionTime[i] - releaseTime[i];
        waitingTime[i] = turnAroundTime[i] - burstTime[i];
    }
     System.out.println("Process ID\tRelease Time\tBrust Time\tCompletion Time\tTurn Around Time\tWaiting Time");
    for (int n = 0; n < PuraProcesses; n++) {
        System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" + burstTime[n] + "\t\t"
            + completionTime[n] + "\t\t" + turnAroundTime[n] + "\t\t\t" + waitingTime[n]);

    }
    System.out.println("\nGantt Chart:");
    int samaye = 0;
```

```
        for (int i = 0; i < PuraProcesses; i++) {
            if (samaye < releaseTime[i]) {
                System.out.print("| Idle ");
                samaye = releaseTime[i];
            }
            System.out.print("| P" + processId[i] + " ");
            samaye += burstTime[i];
        }
        System.out.println("|");
    }
}
```

Output



2. WAP to implement Shortest Job First (SJF) algorithm.

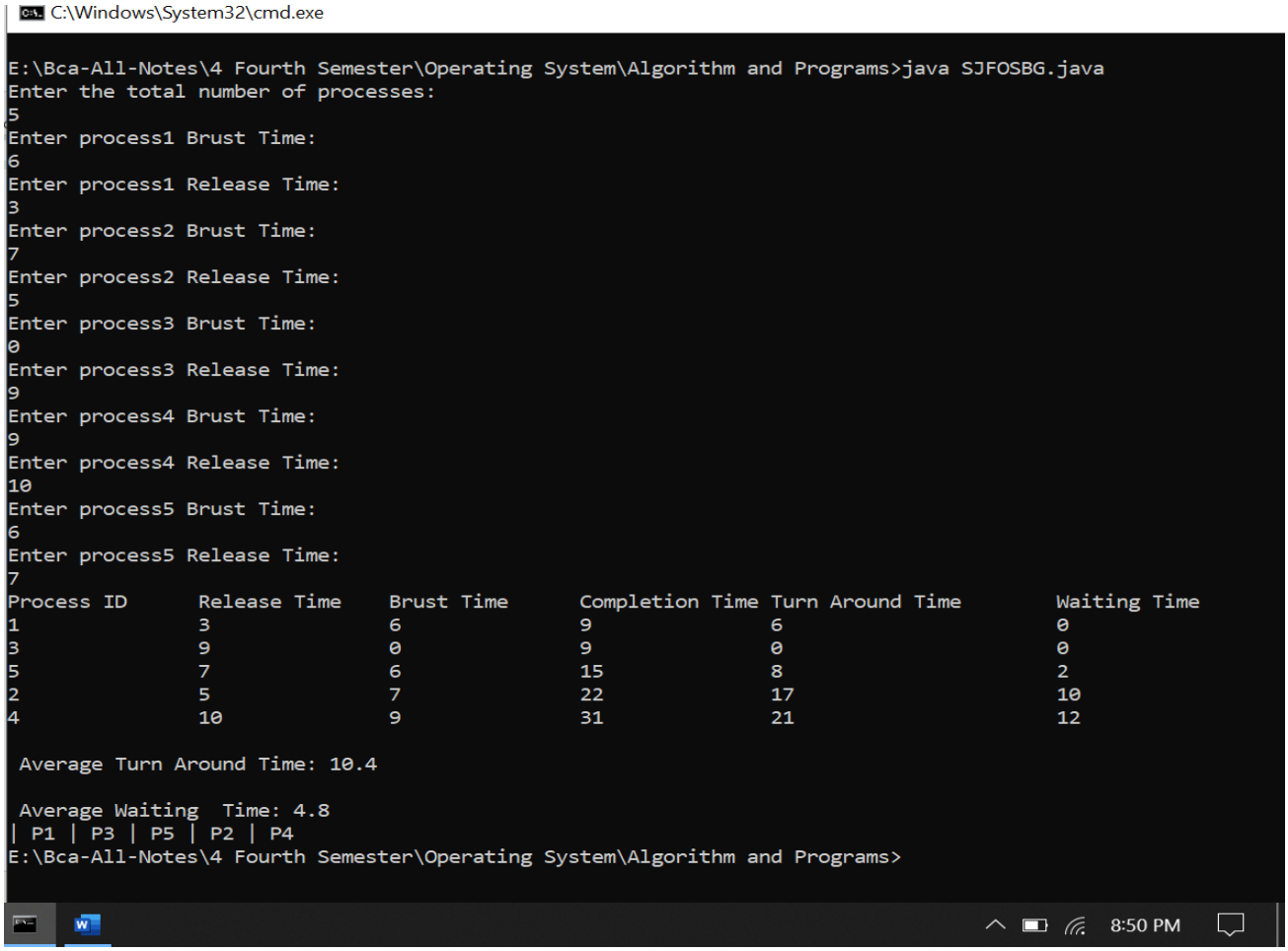
Source code:

```
import java.util.Scanner;
public class SJFOSBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the total number of processes:");
      int PuraProcesses = sc.nextInt();
      int[] processId = new int[PuraProcesses];
      int[] burstTime = new int[PuraProcesses];
      int[] releaseTime = new int[PuraProcesses];
      int[] completionTime = new int[PuraProcesses];
      int[] turnAroundTime = new int[PuraProcesses];
      int[] waitingTime = new int[PuraProcesses];
      int[] processStatus = new int[PuraProcesses];
      int systemTime = 0, finished = 0;
      float avgwt = 0, avgta = 0;
      for (int p = 0; p < PuraProcesses; p++) {
         System.out.println("Enter process" + (p + 1) + " Brust Time:");
         burstTime[p] = sc.nextInt();
         System.out.println("Enter process" + (p + 1) + " Release Time:");
         releaseTime[p] = sc.nextInt();
         processId[p] = p + 1;
         processStatus[p] = 0;
      }
      while (true) {
         int c = PuraProcesses, min = 9999;
         if (finished == PuraProcesses)
            break;
```

```
        for (int i = 0; i < PuraProcesses; i++) {
            if ((releaseTime[i] <= systemTime) && (processStatus[i] == 0) &&
(burstTime[i] < min)) {
                min = burstTime[i];
                c = i;
            }
        }
        if (c == PuraProcesses)
            systemTime++;
        else {
            completionTime[c] = systemTime + burstTime[c];
            systemTime += burstTime[c];
            turnAroundTime[c] = completionTime[c] - releaseTime[c];
            waitingTime[c] = turnAroundTime[c] - burstTime[c];
            processStatus[c] = 1;
            processId[finished] = c + 1;
            finished++;
        }
    }
        System.out.println("Process ID\tRelease Time\tBrust Time\tCompletion
Time\tTurn Around Time\tWaiting Time");
    for (int n = 0; n < PuraProcesses; n++) {
        avgwt += waitingTime[n];
        avgta += turnAroundTime[n];
        int idx = processId[n] - 1; // FIX
        System.out.println(processId[n] + "\t\t" + releaseTime[idx] + "\t\t" +
            burstTime[idx] + "\t\t" + completionTime[idx] + "\t\t" +
            turnAroundTime[idx] + "\t\t\t" + waitingTime[idx]);
    }
```

```
            System.out.println("\n Average Turn Around Time: " + (float) (avgta / PuraProcesses));
        System.out.println("\n Average Waiting  Time: " + (float) (avgwt / PuraProcesses));
         sc.close();
         for (int i = 0; i < PuraProcesses; i++) {
            System.out.print("| P" + processId[i] + " ");
         }
      }
   }
```

Output:



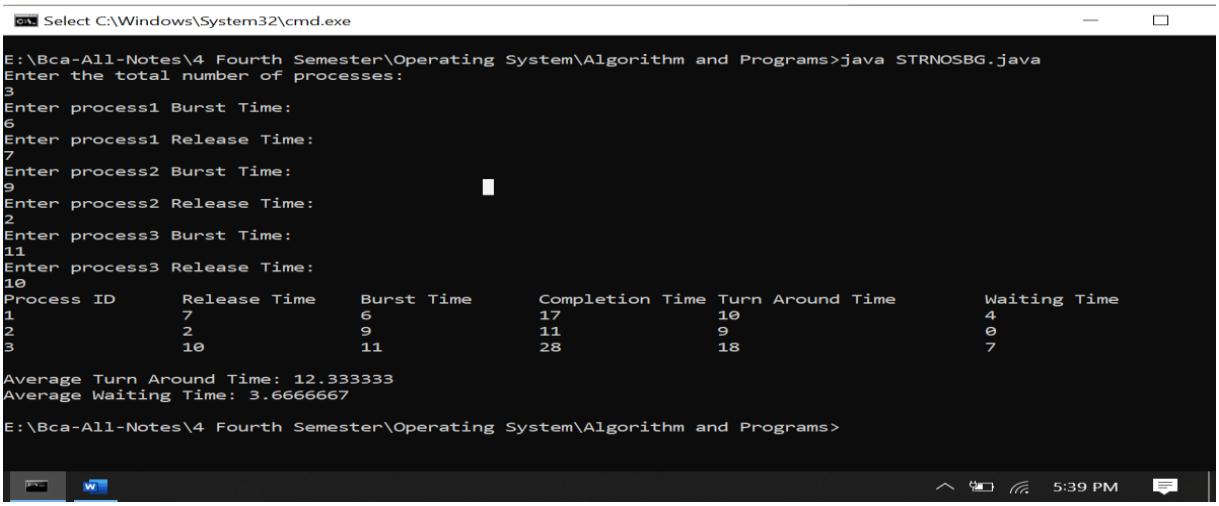
3. WAP to implement Shortest Remaining Time Next algorithm.

Source Code:

```
import java.util.Scanner;
public class STRNOSBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the total number of processes:");
      int PuraProcesses = sc.nextInt();
      int[] processId = new int[PuraProcesses];
      int[] burstTime = new int[PuraProcesses];
      int[] releaseTime = new int[PuraProcesses];
      int[] completionTime = new int[PuraProcesses];
      int[] turnAroundTime = new int[PuraProcesses];
      int[] waitingTime = new int[PuraProcesses];
      int[] remainingTime = new int[PuraProcesses];
      int[] processStatus = new int[PuraProcesses];
      float avgwt = 0, avgta = 0;
      for (int p = 0; p < PuraProcesses; p++) {
         System.out.println("Enter process" + (p + 1) + " Burst Time:");
         burstTime[p] = sc.nextInt();
         System.out.println("Enter process" + (p + 1) + " Release Time:");
         releaseTime[p] = sc.nextInt();  processId[p] = p + 1;
         remainingTime[p] = burstTime[p];   }
      int systemTime = 0, completeProcess = 0;
      while (completeProcess < PuraProcesses) {
         int shortestKamIndex = -1;  int min = 9999;
         for (int i = 0; i < PuraProcesses; i++) {
                  if ((releaseTime[i] <= systemTime) && remainingTime[i] > 0 &&
remainingTime[i] < min) {
               min = remainingTime[i];  shortestKamIndex = i;}}
```

```
            if (shortestKamIndex == -1) {  systemTime++;    continue;  }
            remainingTime[shortestKamIndex]--;  systemTime++;
            if (remainingTime[shortestKamIndex] == 0) {
               completeProcess++;  processStatus[shortestKamIndex] = 1;
               completionTime[shortestKamIndex] = systemTime;
                turnAroundTime[shortestKamIndex] = completionTime[shortestKamIndex]
- releaseTime[shortestKamIndex];
                  waitingTime[shortestKamIndex] = turnAroundTime[shortestKamIndex] -
burstTime[shortestKamIndex];
               avgta += turnAroundTime[shortestKamIndex];
               avgwt += waitingTime[shortestKamIndex];
            } }
                System.out.println("Process ID\tRelease Time\tBurst Time\tCompletion
Time\tTurn Around Time\tWaiting Time");
         for (int n = 0; n < PuraProcesses; n++) {
            System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +
                  burstTime[n] + "\t\t" + completionTime[n] + "\t\t" +
                  turnAroundTime[n] + "\t\t\t" + waitingTime[n]);  }
         System.out.println("\nAverage Turn Around Time: " + (avgta / PuraProcesses));
         System.out.println("Average Waiting Time: " + (avgwt / PuraProcesses));
         sc.close();}}
```

Output:



4. WAP to implement Round Robin algorithm.

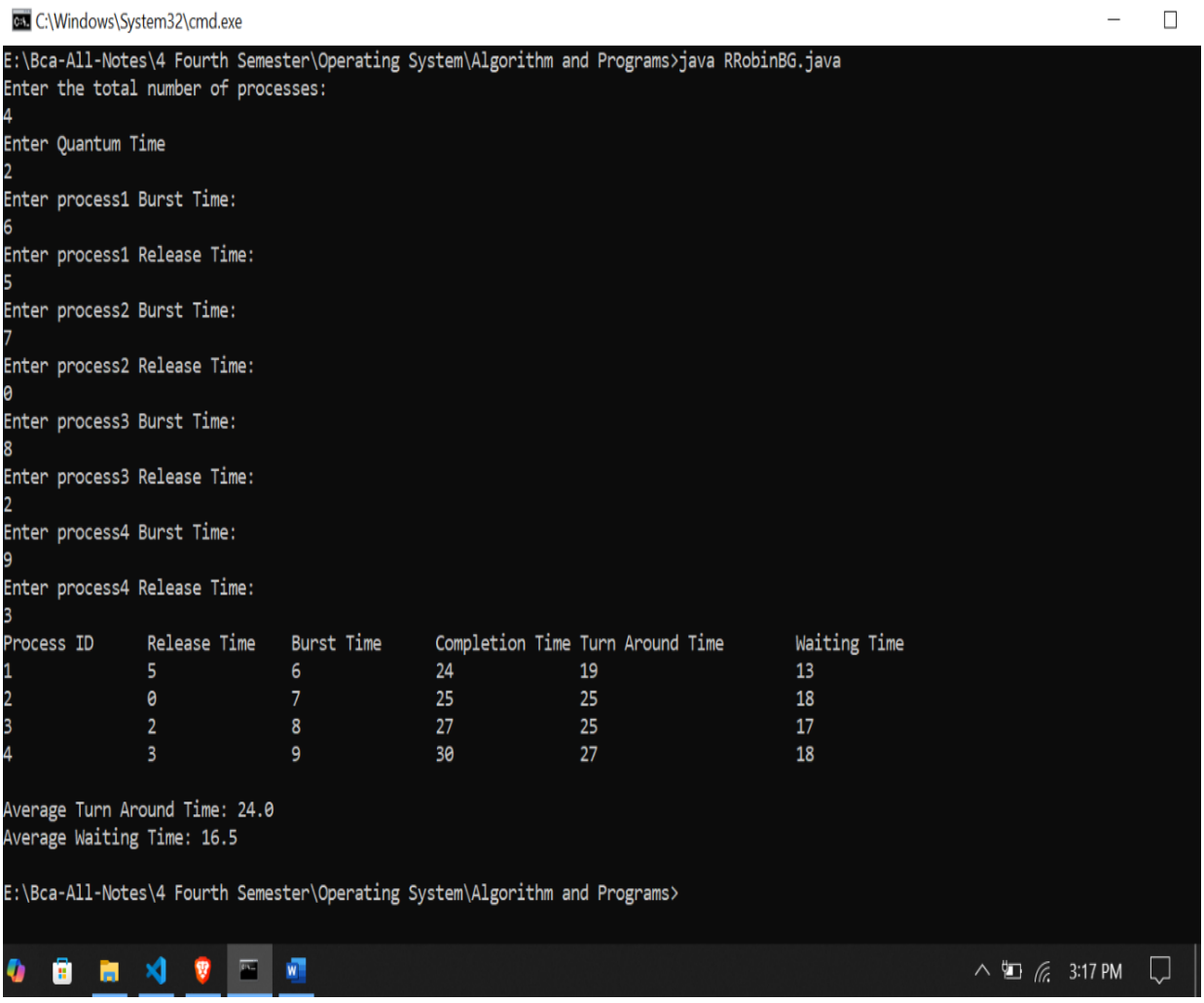
Source Code:

```
import java.util.Scanner;
public class RRobinBG {
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter the total number of processes:");
		int PuraProcesses = sc.nextInt();
		System.out.println("Enter Quantum Time");
		int Quantum = sc.nextInt();
		int[] processId = new int[PuraProcesses];
		int[] burstTime = new int[PuraProcesses];
		int[] releaseTime = new int[PuraProcesses];
		int[] completionTime = new int[PuraProcesses];
		int[] remainingTime = new int[PuraProcesses];

		for (int p = 0; p < PuraProcesses; p++) {
			System.out.println("Enter process" + (p + 1) + " Burst Time:");
			burstTime[p] = sc.nextInt();
			System.out.println("Enter process" + (p + 1) + " Release Time:");
			releaseTime[p] = sc.nextInt();
			processId[p] = p + 1;
			remainingTime[p] = burstTime[p];
		}

		int systemTime = 0, completeProcess = 0;
		float totalTT = 0, totalWT = 0;
		while (completeProcess < PuraProcesses) {
			boolean sakiyo = false;
```

```
                    for (int i = 0; i < PuraProcesses; i++) {
                            if (remainingTime[i] > 0 && releaseTime[i] <= systemTime)
{
                    sakiyo = true;
                    if (remainingTime[i] <= Quantum) {
                    systemTime += remainingTime[i];
                    remainingTime[i] = 0;
                    completionTime[i] = systemTime;
                    completeProcess++;
          } else {
                systemTime += Quantum;
                remainingTime[i] -= Quantum;

                                }
                            }
                    }
                    if (!sakiyo) {
                            systemTime++;
                    }
                }

                System.out.println("Process ID\tRelease Time\tBurst Time\tCompletion
Time\tTurn Around Time\tWaiting Time");
                for (int n = 0; n < PuraProcesses; n++) {
                    int turnAroundTime = completionTime[n] - releaseTime[n];
                    int waitingTime = turnAroundTime - burstTime[n];
                    totalTT += turnAroundTime;
                    totalWT += waitingTime;
                    System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +
                    burstTime[n] + "\t\t" + completionTime[n] + "\t\t" +
                    turnAroundTime + "\t\t\t" + waitingTime);
```

```
                }
                System.out.println("\nAverage Turn Around Time: " + (totalTT / PuraProcesses));
                System.out.println("Average Waiting Time: " + (totalWT / PuraProcesses));
                sc.close();
        }
}
```

Output:



5. WAP to implement Highest Response Ratio Next (HRRN) algorithm.

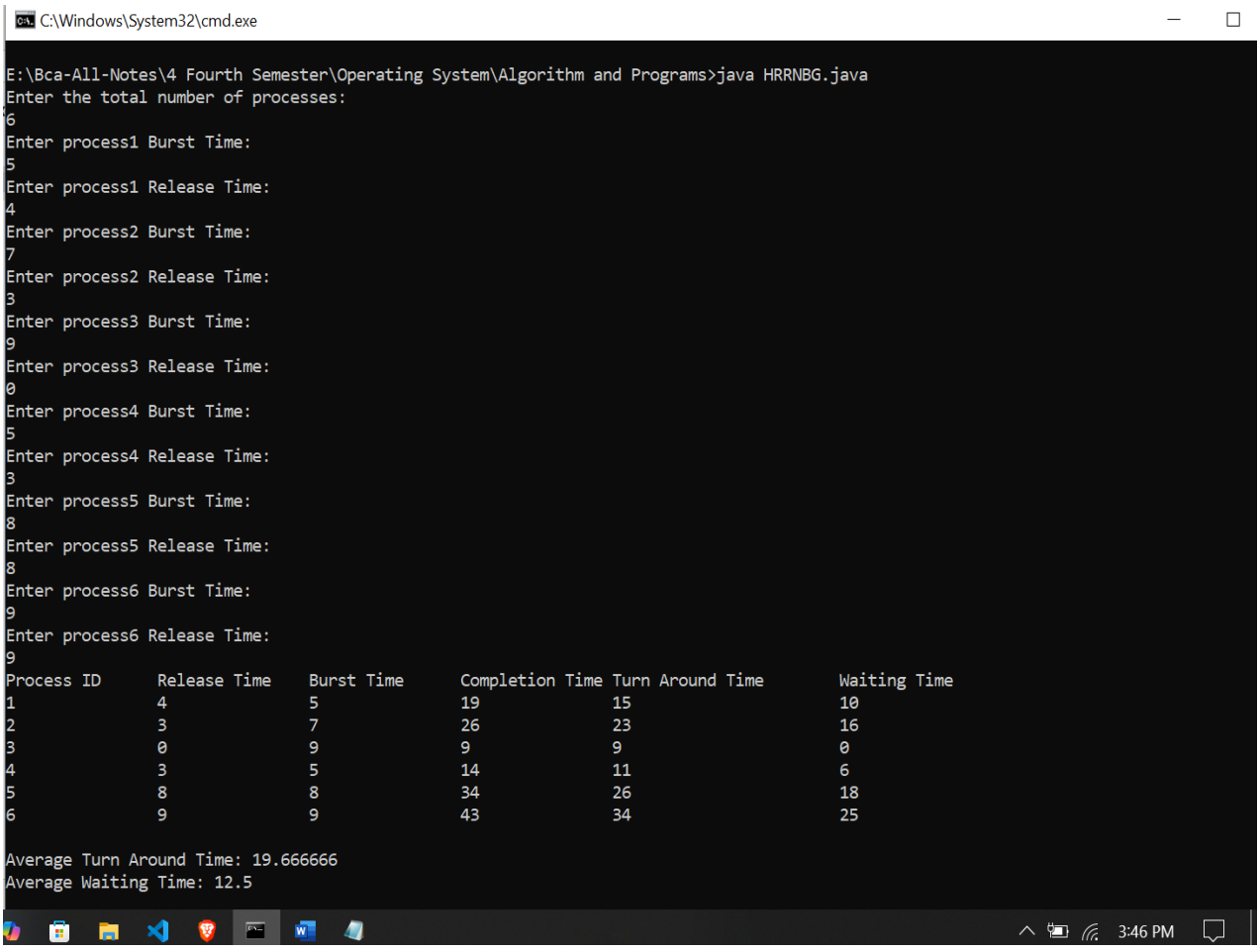
Source Code:

```
import java.util.Scanner;
public class HRRNBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the total number of processes:");
      int PuraProcesses = sc.nextInt();
      int[] processId = new int[PuraProcesses];
      int[] burstTime = new int[PuraProcesses];
      int[] releaseTime = new int[PuraProcesses];
      int[] completionTime = new int[PuraProcesses];
      int[] turnAroundTime = new int[PuraProcesses];
      int[] waitingTime = new int[PuraProcesses];
      boolean[] sakiyo = new boolean[PuraProcesses];

      for (int p = 0; p < PuraProcesses; p++) {
         System.out.println("Enter process" + (p + 1) + " Burst Time:");
         burstTime[p] = sc.nextInt();
         System.out.println("Enter process" + (p + 1) + " Release Time:");
         releaseTime[p] = sc.nextInt();
         processId[p] = p + 1;

      }
      int systemTime = 0, completeProcess = 0;
      float totalTT = 0, totalWT = 0;
      while (completeProcess < PuraProcesses) {
         int thuloResponseIndex = -1;
         float maxResponseRatio = -1;
         for (int i = 0; i < PuraProcesses; i++) {
```

```
            if (!sakiyo[i] && releaseTime[i] <= systemTime) {
                float responseRatio = (float) (systemTime - releaseTime[i] + burstTime[i]) /
burstTime[i];
                if (responseRatio > maxResponseRatio) {
                    maxResponseRatio = responseRatio;
                    thuloResponseIndex = i;
                } else if (responseRatio == maxResponseRatio && thuloResponseIndex != -
1) {
                    if (releaseTime[i] < releaseTime[thuloResponseIndex]) {
                        thuloResponseIndex = i;
                    }
                }
            }
        }
        if (thuloResponseIndex == -1) {
            systemTime++;
            continue;
        } else {
            systemTime += burstTime[thuloResponseIndex];
            completionTime[thuloResponseIndex] = systemTime;
            turnAroundTime[thuloResponseIndex] = completionTime[thuloResponseIndex]
                    - releaseTime[thuloResponseIndex];
            waitingTime[thuloResponseIndex] = turnAroundTime[thuloResponseIndex] -
burstTime[thuloResponseIndex];
            totalTT += turnAroundTime[thuloResponseIndex];
            totalWT += waitingTime[thuloResponseIndex];
            sakiyo[thuloResponseIndex] = true;
            completeProcess++;
        }
    }
    System.out.println("Process ID\tRelease Time\tBurst Time\tCompletion Time\tTurn
Around Time\tWaiting Time");
```

```
        for (int n = 0; n < PuraProcesses; n++) {

            System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +
                    burstTime[n] + "\t\t" + completionTime[n] + "\t\t" +
                    turnAroundTime[n] + "\t\t\t" + waitingTime[n]);
        }
        System.out.println("\nAverage Turn Around Time: " + (totalTT / PuraProcesses));
        System.out.println("Average Waiting Time: " + (totalWT / PuraProcesses));
        sc.close();
    }
}
```

Output:



6. WAP to implement non-preemptive priority scheduling algorithm.

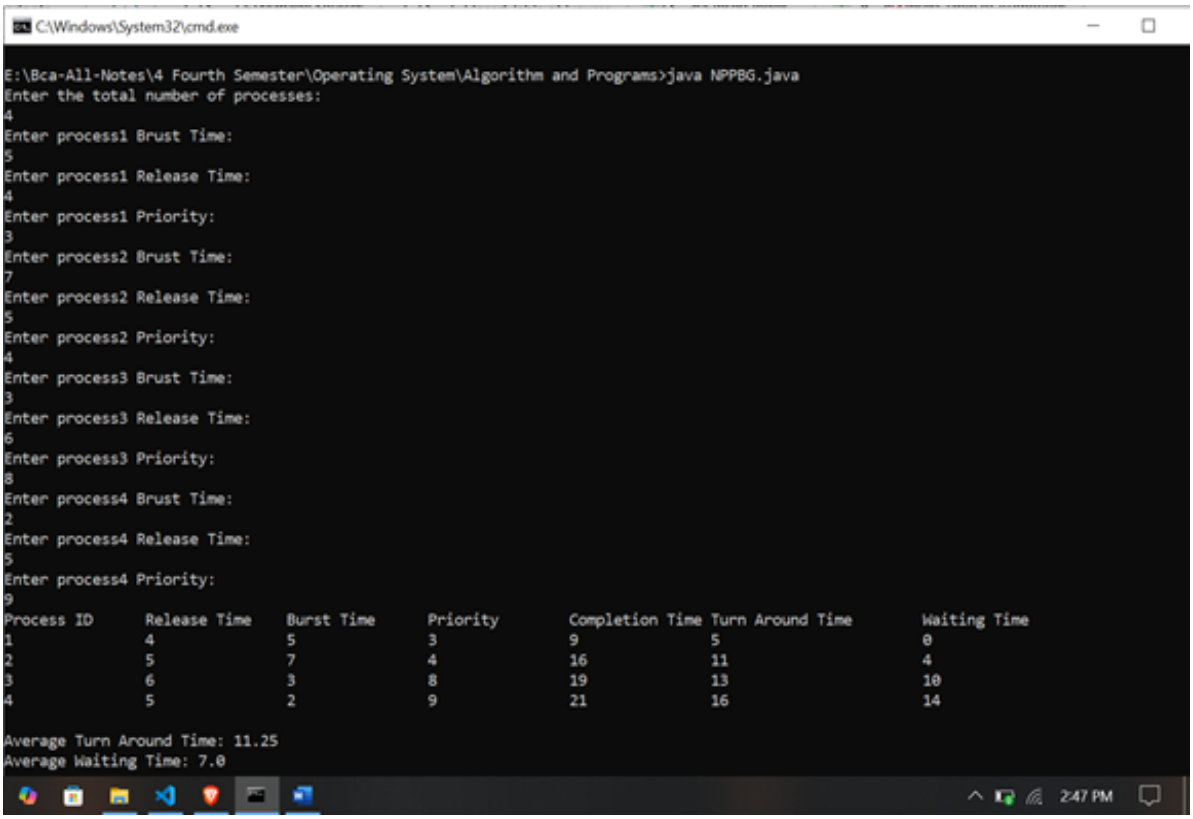
Source Code:

```
import java.util.Scanner;


public class NPPBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the total number of processes:");
      int PuraProcesses = sc.nextInt();
      int[] processId = new int[PuraProcesses];
      int[] burstTime = new int[PuraProcesses];
      int[] releaseTime = new int[PuraProcesses];
      int[] completionTime = new int[PuraProcesses];
      int[] turnAroundTime = new int[PuraProcesses];
      int[] waitingTime = new int[PuraProcesses];
      int[] processPriority = new int[PuraProcesses];
      boolean[] sakiyo = new boolean[PuraProcesses];
      for (int p = 0; p < PuraProcesses; p++) {
         System.out.println("Enter process" + (p + 1) + " Brust Time:");
         burstTime[p] = sc.nextInt();
         System.out.println("Enter process" + (p + 1) + " Release Time:");
         releaseTime[p] = sc.nextInt();
         System.out.println("Enter process" + (p + 1) + " Priority:");
         processPriority[p] = sc.nextInt();
         processId[p] = p + 1;


      }
      int systemTime = 0, finished = 0;
      float totalTAT = 0, totalWT = 0;
```

```
        while (finished < PuraProcesses) {
            int thuloPriorityIndex = -1;
            int thuloPriority = 9999;

            for (int i = 0; i < PuraProcesses; i++) {
                if (!sakiyo[i] && releaseTime[i] <= systemTime && processPriority[i] < thuloPriority) {
                    thuloPriority = processPriority[i];
                    thuloPriorityIndex = i;
                }
            }
            if (thuloPriorityIndex == -1) {
                systemTime++;
                continue;

            }

            systemTime += burstTime[thuloPriorityIndex];
            completionTime[thuloPriorityIndex] = systemTime;
            turnAroundTime[thuloPriorityIndex] = completionTime[thuloPriorityIndex] - releaseTime[thuloPriorityIndex];
            waitingTime[thuloPriorityIndex] = turnAroundTime[thuloPriorityIndex] - burstTime[thuloPriorityIndex];

            totalTAT += turnAroundTime[thuloPriorityIndex];
            totalWT += waitingTime[thuloPriorityIndex];
            sakiyo[thuloPriorityIndex] = true;
            finished++;

        }
```

```
        System.out.println(
                "Process ID\tRelease Time\tBurst Time\tPriority\tCompletion Time\tTurn Around Time\tWaiting Time");
        for (int n = 0; n < PuraProcesses; n++) {
            System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +
                    burstTime[n] + "\t\t" + processPriority[n] + "\t\t" + completionTime[n] + "\t\t" +
                    turnAroundTime[n] + "\t\t\t" + waitingTime[n]);
        }
        System.out.println("\nAverage Turn Around Time: " + (totalTAT / PuraProcesses));
        System.out.println("Average Waiting Time: " + (totalWT / PuraProcesses));
        sc.close();
    }
}
```

Output:



7. WAP to implement preemptive priority scheduling algorithm.

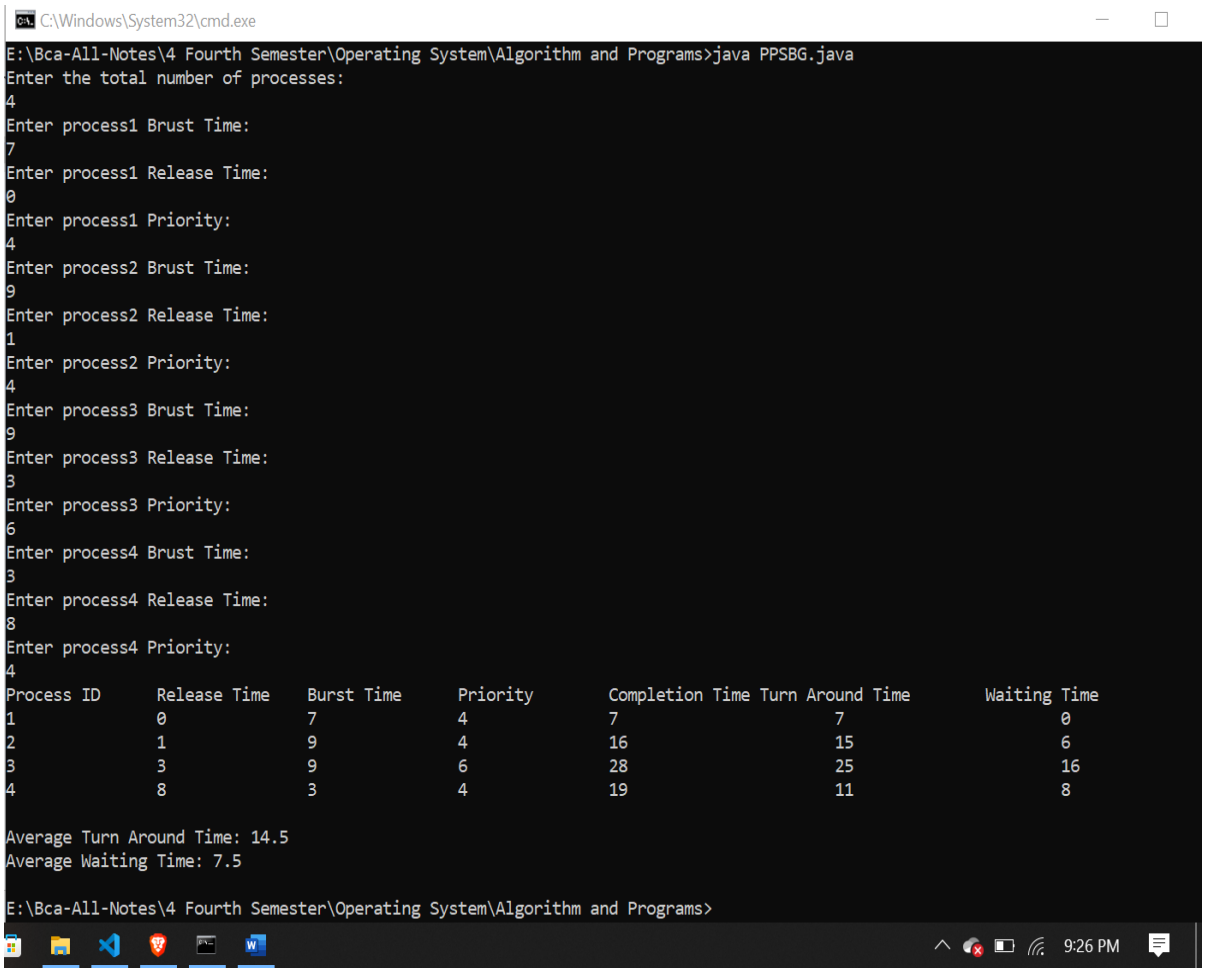
Source code:

```
import java.util.Scanner;
public class PPSBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the total number of processes:");
      int PuraProcesses = sc.nextInt();
      int[] processId = new int[PuraProcesses];
      int[] burstTime = new int[PuraProcesses];
      int[] releaseTime = new int[PuraProcesses];
      int[] completionTime = new int[PuraProcesses];
      int[] turnAroundTime = new int[PuraProcesses];
      int[] waitingTime = new int[PuraProcesses];
      int[] remaningTime = new int[PuraProcesses];
      int[] processPriority = new int[PuraProcesses];
      boolean[] sakiyo = new boolean[PuraProcesses];
      for (int p = 0; p < PuraProcesses; p++) {
         System.out.println("Enter process" + (p + 1) + " Brust Time:");
         burstTime[p] = sc.nextInt();
         remaningTime[p] = burstTime[p];
         System.out.println("Enter process" + (p + 1) + " Release Time:");
         releaseTime[p] = sc.nextInt();
         System.out.println("Enter process" + (p + 1) + " Priority:");
         processPriority[p] = sc.nextInt();
         processId[p] = p + 1;
      }
      int systemTime = 0, finished = 0;
      float totalTAT = 0, totalWT = 0;
```

```
        while (finished < PuraProcesses) {
            int thuloPriorityIndex = -1;
            int thuloPriority = 9999;
            for (int i = 0; i < PuraProcesses; i++) {
                if (!sakiyo[i] && releaseTime[i] <= systemTime && processPriority[i] <
thuloPriority
                    && remaningTime[i] > 0) {
                thuloPriority = processPriority[i];
                thuloPriorityIndex = i;
              }
            }
            if (thuloPriorityIndex == -1) {
              systemTime++;
              continue;
            }
            remaningTime[thuloPriorityIndex]--;
            systemTime++;
            if (remaningTime[thuloPriorityIndex] == 0) {
              completionTime[thuloPriorityIndex] = systemTime;
              turnAroundTime[thuloPriorityIndex] = completionTime[thuloPriorityIndex]
                  - releaseTime[thuloPriorityIndex];
                waitingTime[thuloPriorityIndex] = turnAroundTime[thuloPriorityIndex] -
burstTime[thuloPriorityIndex];
              totalTAT += turnAroundTime[thuloPriorityIndex];
              totalWT += waitingTime[thuloPriorityIndex];
              sakiyo[thuloPriorityIndex] = true;
              finished++;
            }
        }
        System.out.println(
```

```
            "Process ID\tRelease Time\tBurst Time\tPriority\tCompletion Time\tTurn Around
Time\tWaiting Time");
        for (int n = 0; n < PuraProcesses; n++) {
            System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +
                burstTime[n] + "\t\t" + processPriority[n] + "\t\t" + completionTime[n] + "\t\t\t"
+
                 turnAroundTime[n] + "\t\t\t" + waitingTime[n]);
        }
        System.out.println("\nAverage Turn Around Time: " + (totalTAT / PuraProcesses));
        System.out.println("Average Waiting Time: " + (totalWT / PuraProcesses));
        sc.close();
    }
}
```

Output:



Page Replacement Algorithm

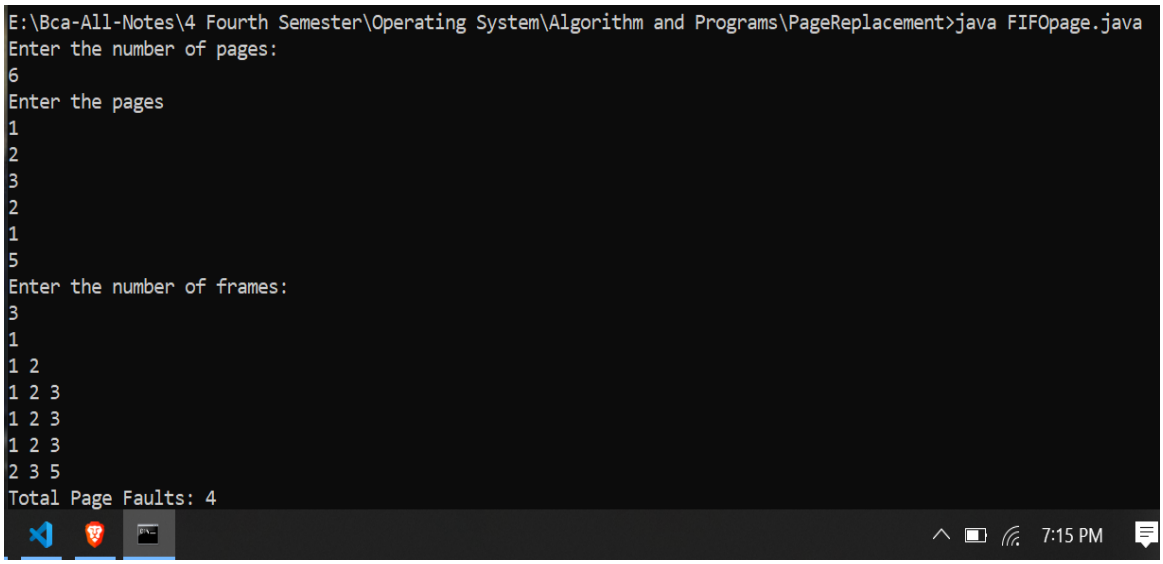
1. WAP to implement First in First Out page replacement algorithm.

Source Code:

```
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;
public class FIFOpage {
   static int pageFault(int pages[], int puraPages, int frameSize) {
      HashSet<Integer> s = new HashSet<>(frameSize);
      Queue<Integer> indexes = new LinkedList<>();
      int pageFaults = 0;
      for (int i = 0; i < puraPages; i++) {
         if (s.size() < frameSize) {
            if (!s.contains(pages[i])) {
               s.add(pages[i]);
               pageFaults++;
               indexes.add(pages[i]);
            }
         } else {
            if (!s.contains(pages[i])) {
               int val = indexes.poll();
               s.remove(val);
               s.add(pages[i]);
               indexes.add(pages[i]);
               pageFaults++;
            }
         }
         for (int page : indexes) {
```

```
            System.out.print(page + " ");
        }System.out.println();
    }     return pageFaults;
  }
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter the number of pages:");
    int puraPages = sc.nextInt();
    int pages[] = new int[puraPages];
    System.out.println("Enter the pages ");
    for (int p = 0; p < puraPages; p++) {
      pages[p] = sc.nextInt();
    }
    System.out.println("Enter the number of frames:");
    int frameSize = sc.nextInt();
    int totalPageFaults = pageFault(pages, puraPages, frameSize);
    System.out.println("Total Page Faults: " + totalPageFaults);
  }
}
```

Output:



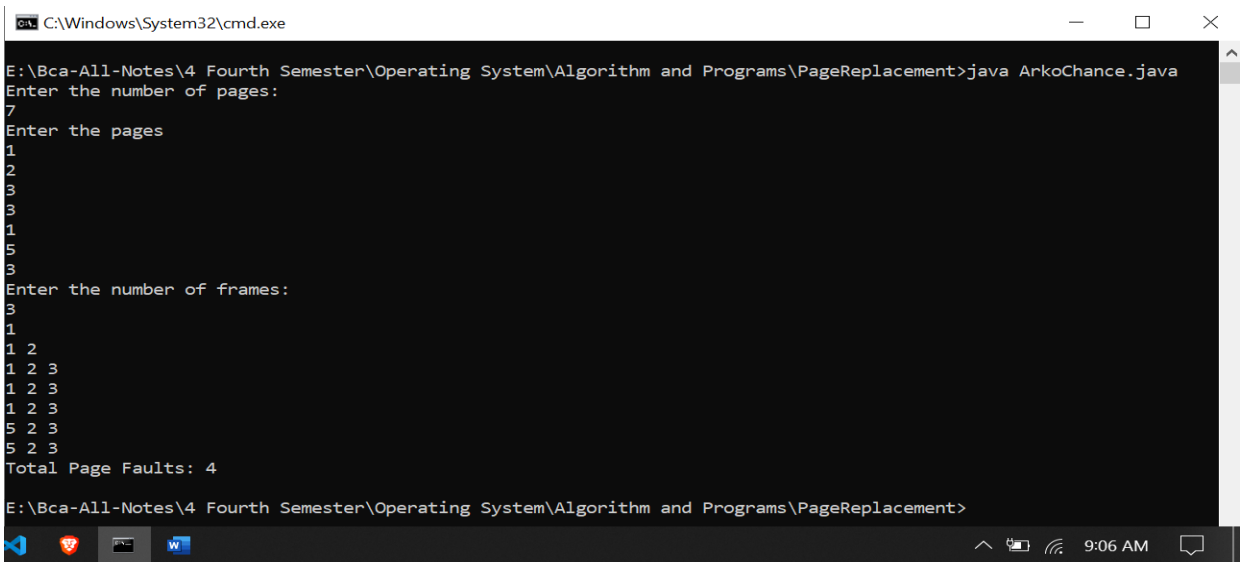
2. WAP to implement second chance page replacement algorithm.

Source Code:

```
import java.util.LinkedList;
import java.util.Scanner;
public class ArkoChance {
   static int pageFault(int pages[], int puraPages, int frameSize) {
      LinkedList<Integer> frames = new LinkedList<>();
      boolean[] reference = new boolean[frameSize];
      int pageFaults = 0, pointer = 0;
      for (int i = 0; i < puraPages; i++) {
         int page = pages[i];
         boolean found = false;
         for (int j = 0; j < frames.size(); j++) {
            if (frames.get(j) == page) {
               reference[j] = true;
               found = true;
               break;  }  }
         if (!found) {
            pageFaults++;
            while (true) {
               if (frames.size() < frameSize) {
                  frames.add(page);
                  reference[frames.indexOf(page)] = true;
                  break;
               } else if (reference[pointer]) {
                  reference[pointer] = false;
                  pointer = (pointer + 1) % frameSize;
               } else {
                  frames.set(pointer, page);
                  reference[pointer] = true;
```

```
                    pointer = (pointer + 1) % frameSize;
                    break;  }  }  }
            for (  int f : frames) {
                System.out.print(f + " ");  }
            System.out.println();  }
        return pageFaults;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of pages:");
        int puraPages = sc.nextInt();
        int pages[] = new int[puraPages];
        System.out.println("Enter the pages ");
        for (int p = 0; p < puraPages; p++) {
            pages[p] = sc.nextInt();  }
        System.out.println("Enter the number of frames:");
        int frameSize = sc.nextInt();
        int totalPageFaults = pageFault(pages, puraPages, frameSize);
        System.out.println("Total Page Faults: " + totalPageFaults);
    }}
```

Output:



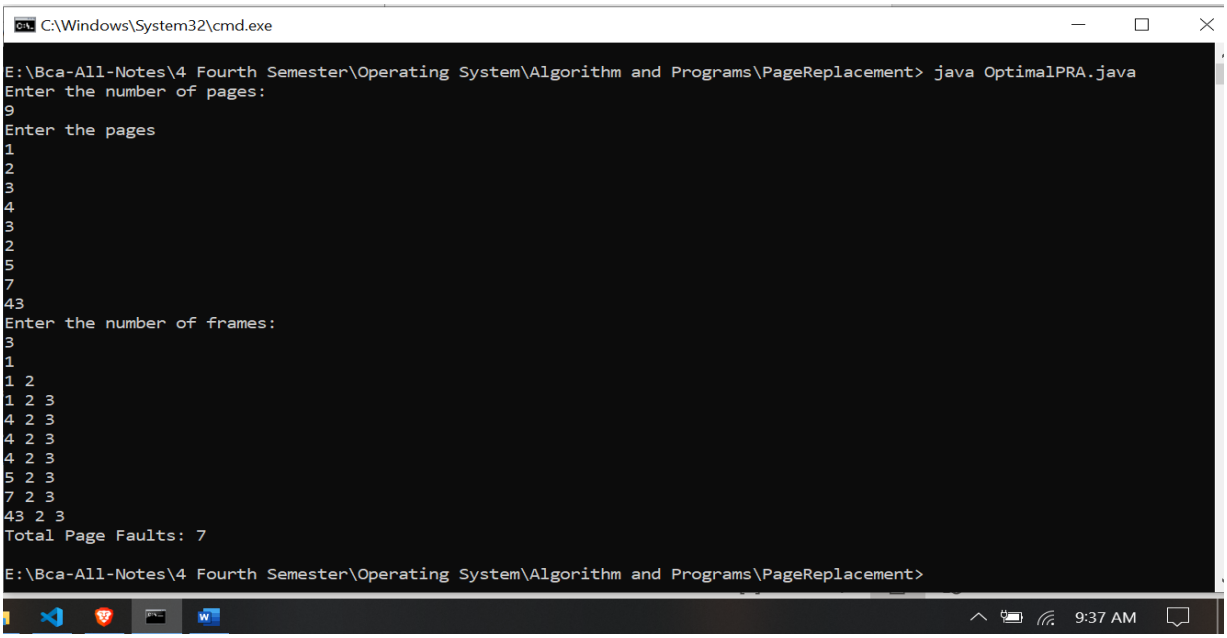
3. WAP to implement optimal page replacement algorithm.

Source Code:

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Scanner;
public class OptimalPRA
   static int pageFault(int pages[], int puraPages, int frameSize) {
      ArrayList<Integer> frames = new ArrayList<>();
      int pageFaults = 0;
      for (int i = 0; i < puraPages; i++) {
         int page = pages[i];
         if (!frames.contains(page)) {
            pageFaults++;
            if (frames.size() == frameSize) {
               int tada = -1, replaceIndex = -1;
               for (int j = 0; j < frames.size(); j++) {
                  int framePage = frames.get(j);
                  int arkoUse = 99999;
                  for (int k = i + 1; k < puraPages; k++) {
                     if (pages[k] == framePage) {
                        arkoUse = k;
                        break;
                     }
                  }
                  if (arkoUse > tada) {
                     tada = arkoUse;
                     replaceIndex = j;      }  }
               frames.set(replaceIndex, page);
            } else {    frames.add(page)  }   }
         for (  int f : frames) {
```

```
        System.out.print(f + " ") }
          System.out.println()   }
      return pageFaults;    }
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of pages:");
      int puraPages = sc.nextInt();
      int pages[] = new int[puraPages];
      System.out.println("Enter the pages ");
      for (int p = 0; p < puraPages; p++) {
          pages[p] = sc.nextInt();
      }
      System.out.println("Enter the number of frames:");
      int frameSize = sc.nextInt();
      int totalPageFaults = pageFault(pages, puraPages, frameSize);
      System.out.println("Total Page Faults: " + totalPageFaults);
  }}
```

Output:



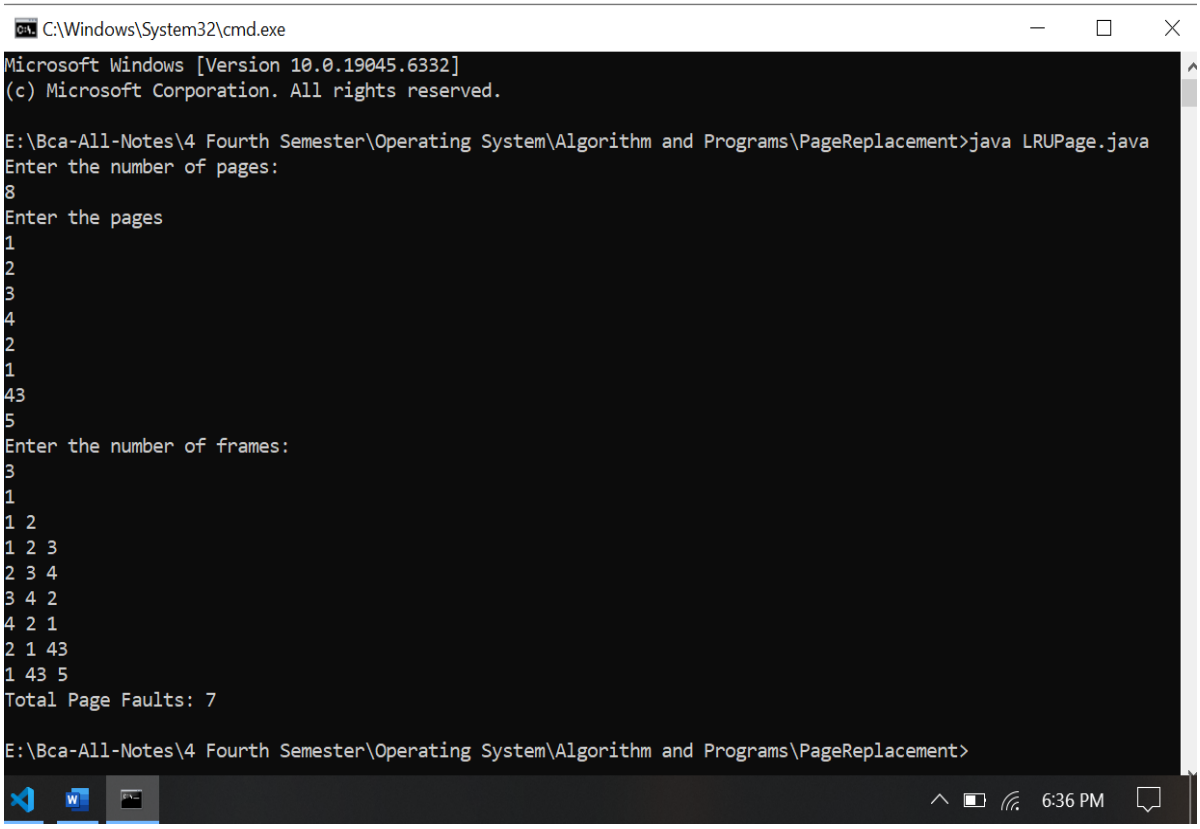
4. WAP to implement Least Recently Used algorithm.

Source Code:

```
import java.util.LinkedList;
import java.util.Scanner;
public class LRUPage {
   static int pageFault(int pages[], int puraPages, int frameSize) {
      LinkedList<Integer> frames = new LinkedList<>();
      int pageFaults = 0;
      for (int i = 0; i < puraPages; i++) {
         int page = pages[i];
         if (!frames.contains(page)) {
            pageFaults++;
            if (frames.size() < frameSize) {
               frames.add(page);
            } else {
               frames.removeFirst();
               frames.add(page);
            }
         } else {
            frames.remove((Integer) page);
            frames.add(page);
         }
         for (int f : frames) {
            System.out.print(f + " ");
         }
         System.out.println();
      }
      return pageFaults;
   }
   public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of pages:");
        int puraPages = sc.nextInt();
        int pages[] = new int[puraPages];
        System.out.println("Enter the pages ");
        for (int p = 0; p < puraPages; p++) {
            pages[p] = sc.nextInt();
        }
        System.out.println("Enter the number of frames:");
        int frameSize = sc.nextInt();
        int totalPageFaults = pageFault(pages, puraPages, frameSize);
        System.out.println("Total Page Faults: " + totalPageFaults);
    }
}
```

Output:



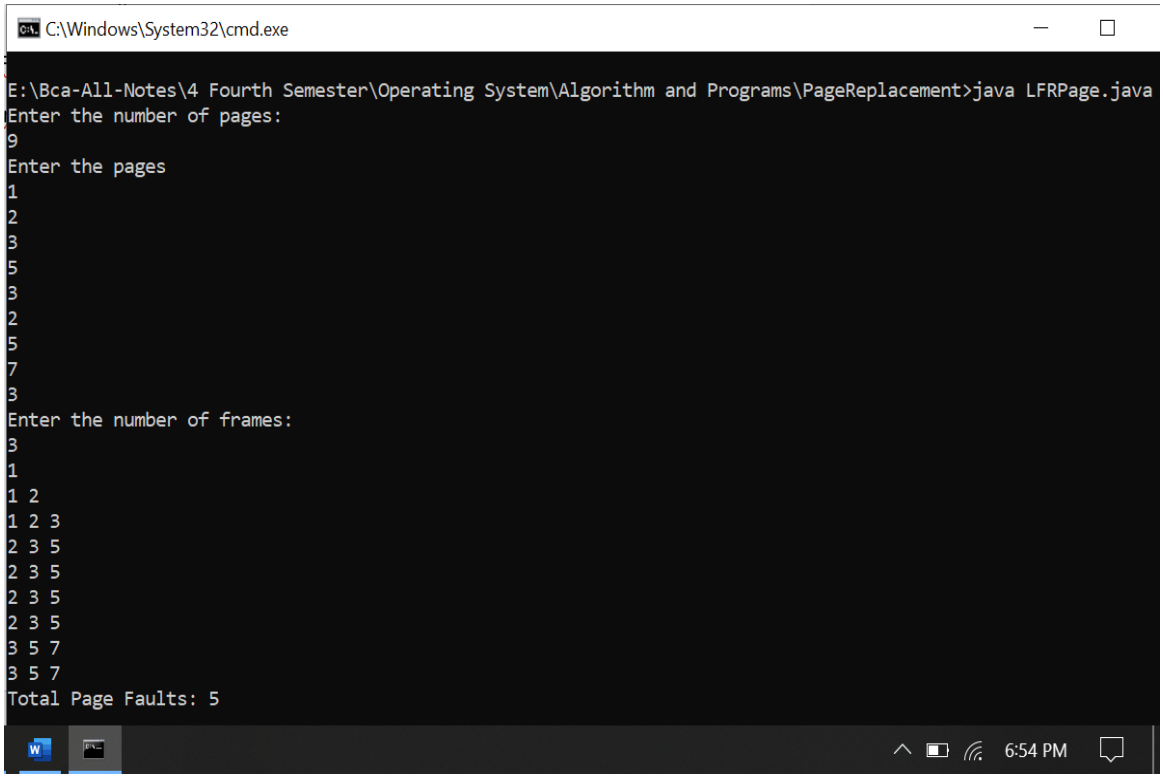
5. WAP to implement Least Frequently Used algorithm.

Source Code:

```
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Scanner;
public class LFRPage {
   static int pageFault(int pages[], int puraPages, int frameSize) {
      HashMap<Integer, Integer> pageFrequency = new HashMap<>();
      LinkedList<Integer> frames = new LinkedList<>();
      int pageFault = 0;
      for (int i = 0; i < puraPages; i++) {
         int page = pages[i];
         if (!frames.contains(page)) {
            pageFault++;
            if (frames.size() < frameSize) {
               frames.add(page);
            } else {
               int lfrPage = -1;
               int minFreq = Integer.MAX_VALUE;
               for (int f : frames) {
                  int freq = pageFrequency.getOrDefault(f, 0);
                  if (freq < minFreq) {
                     minFreq = freq;
                     lfrPage = f;}}
               frames.remove((Integer) lfrPage);
               frames.add(page);   }  }
         pageFrequency.put(page, pageFrequency.getOrDefault(page, 0) + 1);
         for (int f : frames) {
            System.out.print(f + " ");
         }     System.out.println();
```

```
    }      return pageFault;   }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of pages:");
        int puraPages = sc.nextInt();
        int pages[] = new int[puraPages];
        System.out.println("Enter the pages ");
        for (int p = 0; p < puraPages; p++) {
            pages[p] = sc.nextInt();}
        System.out.println("Enter the number of frames:");
        int frameSize = sc.nextInt();
        int totalPageFaults = pageFault(pages, puraPages, frameSize);
        System.out.println("Total Page Faults: " + totalPageFaults);
    }
}
```

Output:



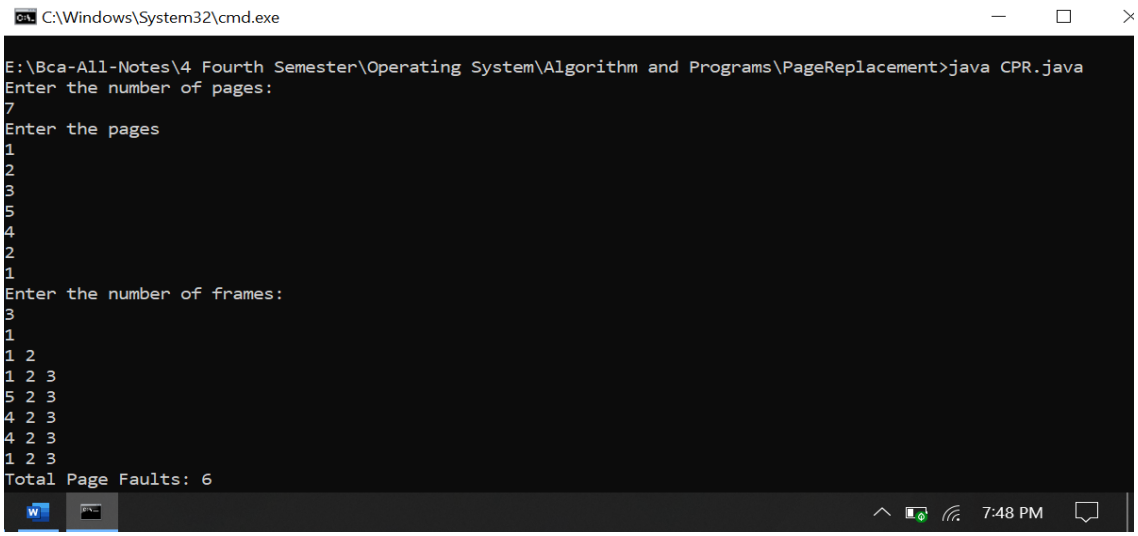
6. WAP to implement Clock Page replacement algorithm.

Source Code:

```
import java.util.LinkedList;
import java.util.Scanner;
public class CPR {
   static int pageFault(int pages[], int puraPages, int frameSize) {
      LinkedList<Integer> frames = new LinkedList<>();
      boolean[] refBits = new boolean[frameSize];
      int pageFault = 0, pointer = 0;
      for (int i = 0; i < puraPages; i++) {
         int page = pages[i];
         boolean pageVetiyo = false;
         for (int j = 0; j < frames.size(); j++) {
            if (frames.get(j) == page) {
               pageVetiyo = true;
               refBits[j] = true;
               break;
}
  }
         if (!pageVetiyo) {
            pageFault++;
            while (true) {
               if (frames.size() < frameSize) {
                  frames.add(page);
                  refBits[frames.indexOf(page)] = true;
                  break;
               } else {
                  if (!refBits[pointer] == false) {
                     frames.set(pointer, page);
                     refBits[pointer] = true;    break;      } else {
                     refBits[pointer] = false;
```

```
                    pointer = (pointer + 1) % frameSize;    } }}  }
            for (int f : frames) {
                System.out.print(f + " ");
}
            System.out.println();  }
        return pageFault;  }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of pages:");
        int puraPages = sc.nextInt();
        int pages[] = new int[puraPages];
        System.out.println("Enter the pages ");
        for (int p = 0; p < puraPages; p++) {
            pages[p] = sc.nextInt();  }
        System.out.println("Enter the number of frames:");
        int frameSize = sc.nextInt();
        int totalPageFaults = pageFault(pages, puraPages, frameSize);
        System.out.println("Total Page Faults: " + totalPageFaults);
    }
}
```

Output:



Disk Scheduling Algorithm

1. WAP to implement First Come First Serve algorithm.

Source Code:

```
import java.util.Scanner;
public class FfcsdiskBg {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.println("Enter the number of requests:");
       int binti = sc.nextInt();
       int requests[] = new int[binti];
       System.out.println("Enter the disk sequence:");
       for (int b = 0; b < binti; b++) {
 requests[b] = sc.nextInt();}
       System.out.println("Enter the initial head position:");
       int suruPosition = sc.nextInt();
       int totalHeadMovement = 0;
       int ailekoPosition = suruPosition;
       System.out.println("Disk head movements:");
       for (int m = 0; m < binti; m++) {
          System.out.println("Moving From " + ailekoPosition + " to " + requests[m]);
          totalHeadMovement += Math.abs(ailekoPosition - requests[m]);
          ailekoPosition = requests[m]; }
       System.out.println("Total head movement is: " + totalHeadMovement);
}}
```

Output:

```
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DiskScheduling>java FfcsdiskBg.java
Enter the number of requests:
5
Enter the disk sequence:
12
34
2
66
90
Enter the initial head position:
2
Disk head movements:
Moving From 2 to 12
Moving From 12 to 34
Moving From 34 to 2
Moving From 2 to 66
Moving From 66 to 90
Total head movement is: 152
```

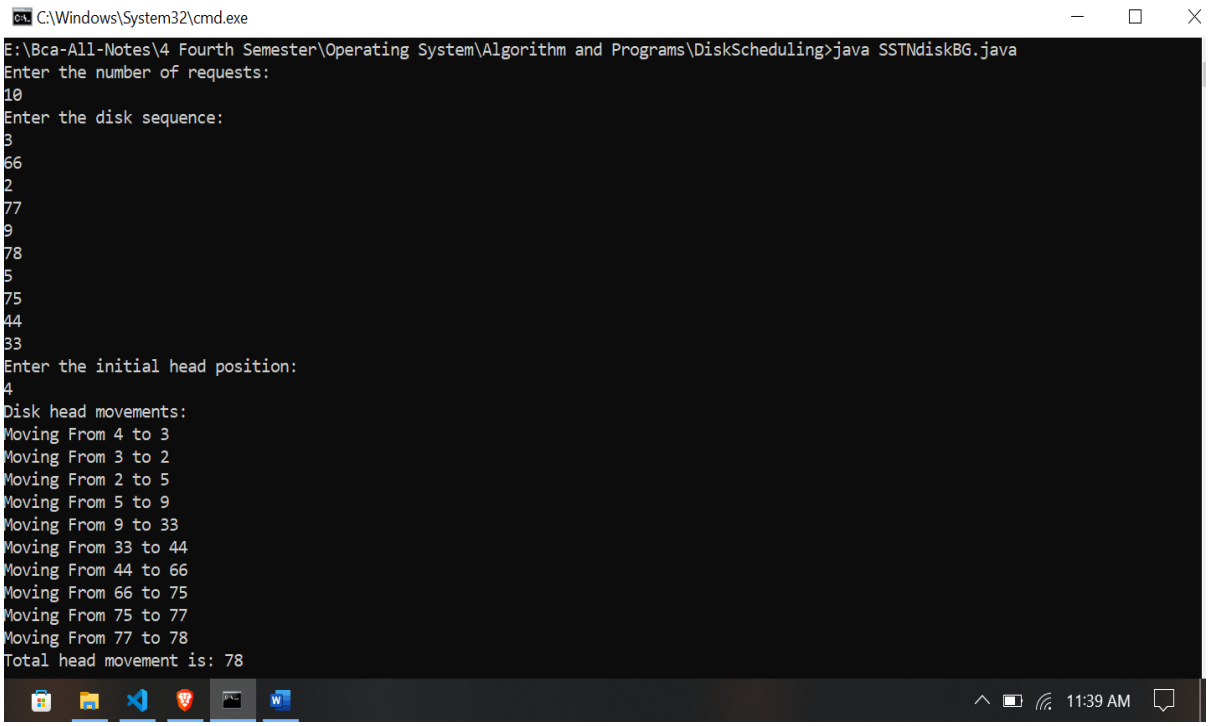
2. WAP to implement Shortest Seek Time First algorithm.

Source Code:

```
import java.util.Scanner;

public class SSTNdiskBg {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of requests:");
      int binti = sc.nextInt();
      int requests[] = new int[binti];
      System.out.println("Enter the disk sequence:");
      for (int b = 0; b < binti; b++) {
         requests[b] = sc.nextInt();
      }
      System.out.println("Enter the initial head position:");
      int suruPosition = sc.nextInt();
      boolean visited[] = new boolean[binti];
      int totalHeadMovement = 0;
      int ailekoPosition = suruPosition;
      System.out.println("Disk head movements:");
      for (int m = 0; m < binti; m++) {
         int thoriiDistance = Integer.MAX_VALUE;
         int index = -1;
         for (int j = 0; j < binti; j++) {
            if (!visited[j]) {
               int distance = Math.abs(ailekoPosition - requests[j]);
               if (distance < thoriiDistance) {
                  thoriiDistance = distance;
                  index = j;
```

```
                }
            }
        }
        visited[index] = true;
        System.out.println("Moving From " + ailekoPosition + " to " + requests[index]);
        totalHeadMovement += thoriiDistance;
        ailekoPosition = requests[index];
    }
    System.out.println("Total head movement is: " + totalHeadMovement);
  }
}
```

Output:



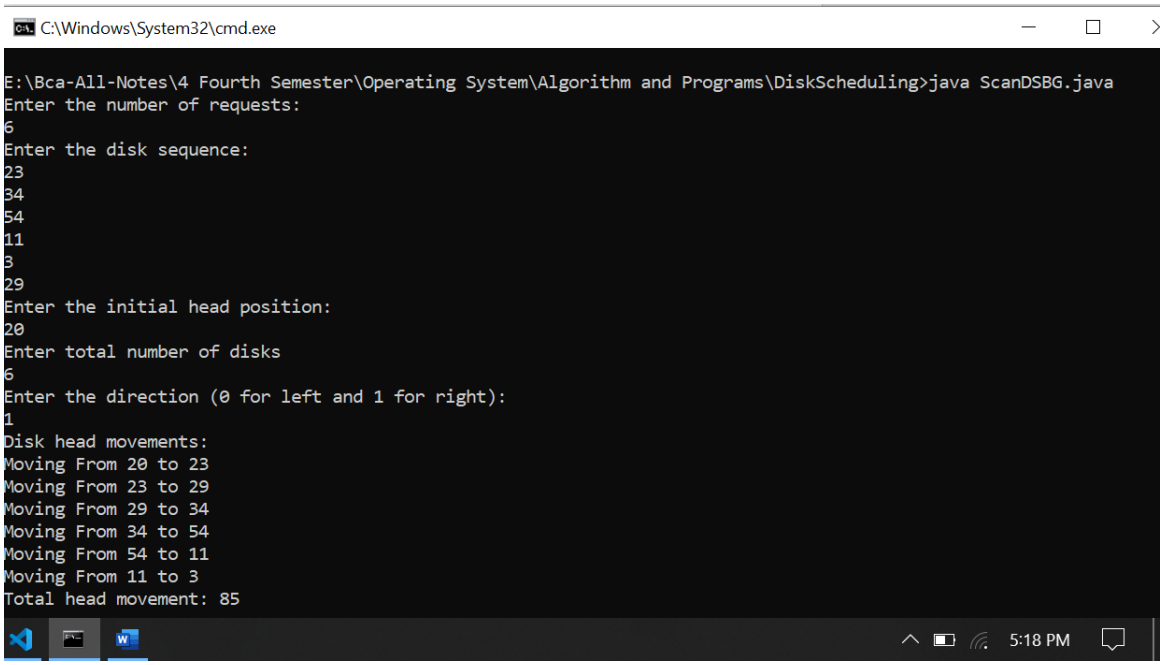
3. WAP to implement Scan Disk Scheduling Algorithm.

Source Code:

```
import java.util.Arrays;
import java.util.Scanner;
public class ScanDSBG {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.println("Enter the number of requests:");
       int binti = sc.nextInt();
       int requests[] = new int[binti];
       System.out.println("Enter the disk sequence:");
       for (int b = 0; b < binti; b++) {requests[b] = sc.nextInt();}
       System.out.println("Enter the initial head position:");
       int suruPosition = sc.nextInt();
       System.out.println("Enter total number of disks");
       int totalDisks = sc.nextInt();
       System.out.println("Enter the direction (0 for left and 1 for right):");
       int direction = sc.nextInt();
       Arrays.sort(requests);
int totalHeadMovement = 0;    int ailekoPosition = suruPosition;
       System.out.println("Disk head movements:");
       if (direction == 1) {  for (int j = 0; j < binti; j++) {
  if (requests[j] >= suruPosition) {
                System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
                ailekoPosition = requests[j];    }  }
          if (ailekoPosition < totalDisks - 1) {
             System.out.println("Moving From " + ailekoPosition + " to " + (totalDisks - 1));
             totalHeadMovement += Math.abs(ailekoPosition - (totalDisks - 1));
             ailekoPosition = totalDisks - 1;}
          for (int k = binti - 1; k >= 0; k--) {    if (requests[k] < suruPosition) {
```

```
            System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
            ailekoPosition = requests[k];  }  }  } else {
        for (int j = binti - 1; j >= 0; j--) {  if (requests[j] <= suruPosition) {
            System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
            ailekoPosition = requests[j];  }}  if (ailekoPosition > 0) {
          System.out.println("Moving From " + ailekoPosition + " to " + 0);
          totalHeadMovement += Math.abs(ailekoPosition - 0);
          ailekoPosition = 0;  }
        for (int k = 0; k < binti; k++) {  if (requests[k] > suruPosition) {
            System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
            ailekoPosition = requests[k];  }    }  }
      System.out.println("Total head movement: " + totalHeadMovement);
      sc.close();  }
}
```

Output:



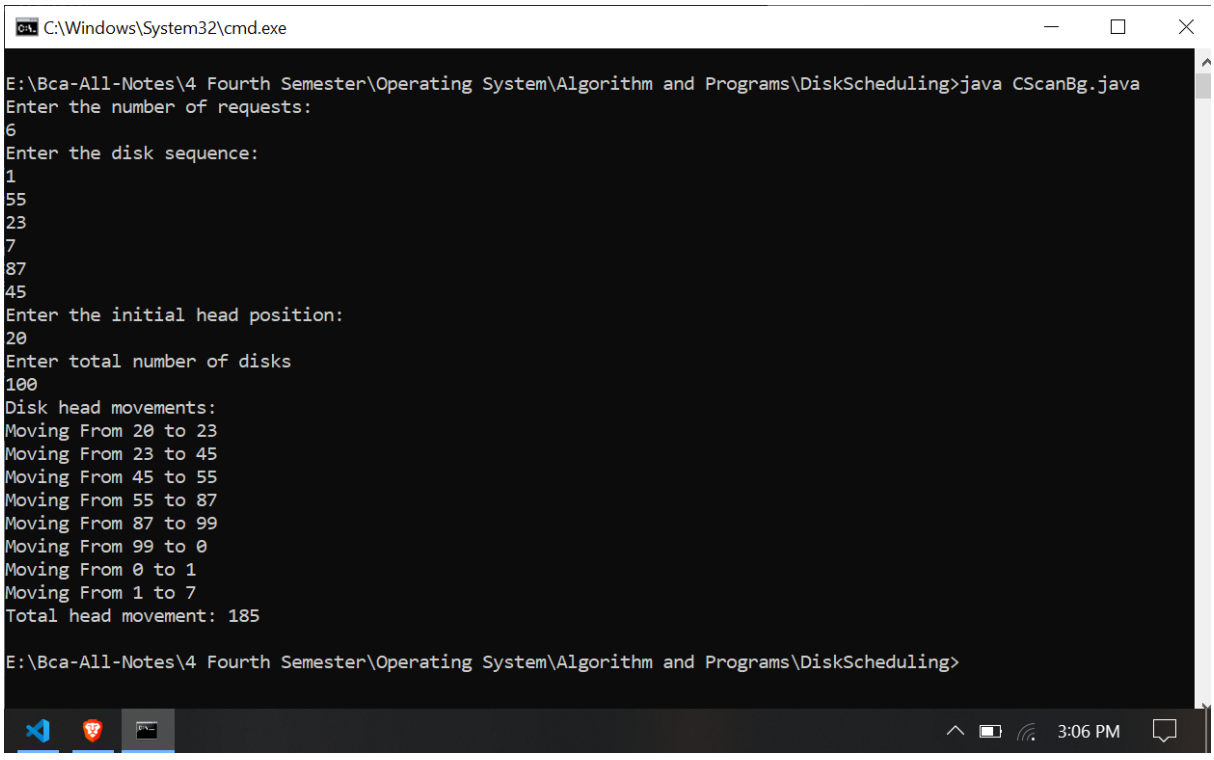
4. WAP to implement C-Scan Disk Scheduling Algorithm.

Source Code:

```
import java.util.Arrays;
import java.util.Scanner;
public class CScanBg {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of requests:");
      int binti = sc.nextInt();
      int requests[] = new int[binti];
      System.out.println("Enter the disk sequence:");
      for (int b = 0; b < binti; b++) {
         requests[b] = sc.nextInt();
      }
      System.out.println("Enter the initial head position:");
      int suruPosition = sc.nextInt();
      System.out.println("Enter total number of disks");
      int totalDisks = sc.nextInt();
      Arrays.sort(requests);
      int totalHeadMovement = 0;
      int ailekoPosition = suruPosition;
      System.out.println("Disk head movements:");
      for (int j = 0; j < binti; j++) {
         if (requests[j] >= suruPosition) {
            System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
            ailekoPosition = requests[j];
         }
      }
      if (ailekoPosition < totalDisks - 1) {
         System.out.println("Moving From " + ailekoPosition + " to " + (totalDisks - 1));
```

```
            totalHeadMovement += Math.abs(ailekoPosition - (totalDisks - 1));
            ailekoPosition = totalDisks - 1;
        }
        System.out.println("Moving From " + (totalDisks - 1) + " to 0");
    totalHeadMovement += (totalDisks - 1);
        ailekoPosition = 0;
        for (int k = 0; k < binti; k++) {
            if (requests[k] < suruPosition) {
                System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
                ailekoPosition = requests[k];
            }
        }
        System.out.println("Total head movement: " + totalHeadMovement);
    }
}
```

Output:



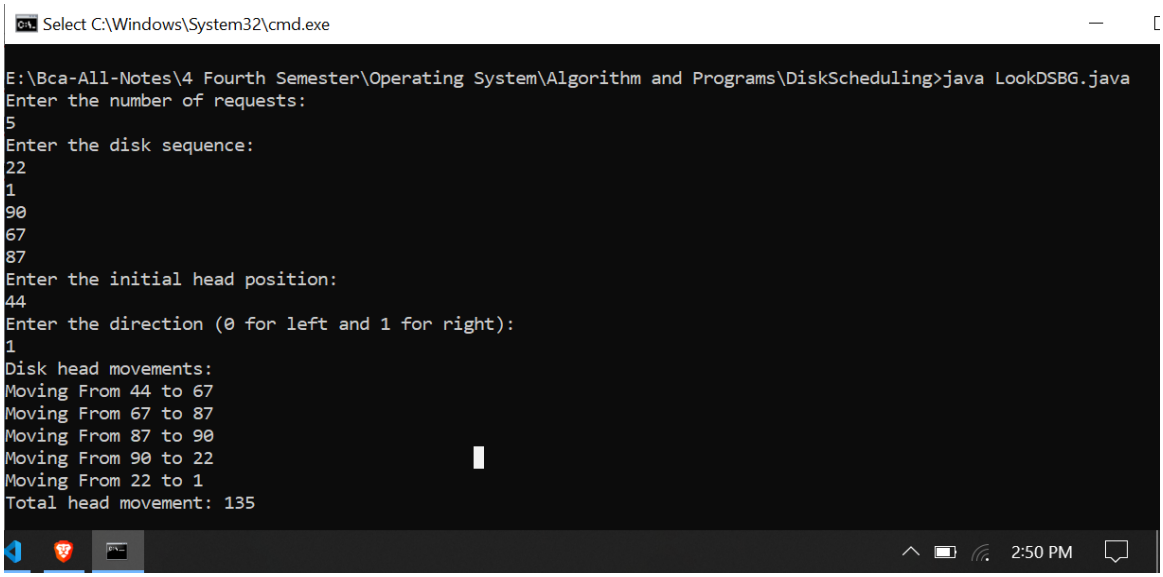
5. WAP to implement Look Disk Scheduling Algorithm.

Source code:

```
import java.util.Arrays;
import java.util.Scanner;
public class LookDSBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of requests:");
      int binti = sc.nextInt();
      int requests[] = new int[binti];
      System.out.println("Enter the disk sequence:");
      for (int b = 0; b < binti; b++) {
         requests[b] = sc.nextInt();  }
      System.out.println("Enter the initial head position:");
      int suruPosition = sc.nextInt();
      System.out.println("Enter the direction (0 for left and 1 for right):");
      int direction = sc.nextInt();
      Arrays.sort(requests);
      int totalHeadMovement = 0;
      int ailekoPosition = suruPosition;
      System.out.println("Disk head movements:");
      if (direction == 1) {
         for (int j = 0; j < binti; j++) {
            if (requests[j] >= suruPosition) {
               System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
               totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
               ailekoPosition = requests[j];

   }     }
         for (int k = binti - 1; k >= 0; k--) {
            if (requests[k] < suruPosition) {
```

```
                System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
                ailekoPosition = requests[k];  }
 }}
else {
        for (int j = binti - 1; j >= 0; j--) {
           if (requests[j] <= suruPosition) {
                System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
                ailekoPosition = requests[j];}
}
        for (int k = 0; k < binti; k++) {
           if (requests[k] > suruPosition) {
                System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
                ailekoPosition = requests[k];  }    }       }
      System.out.println("Total head movement: " + totalHeadMovement);
      sc.close();
 }
}
```

Output:



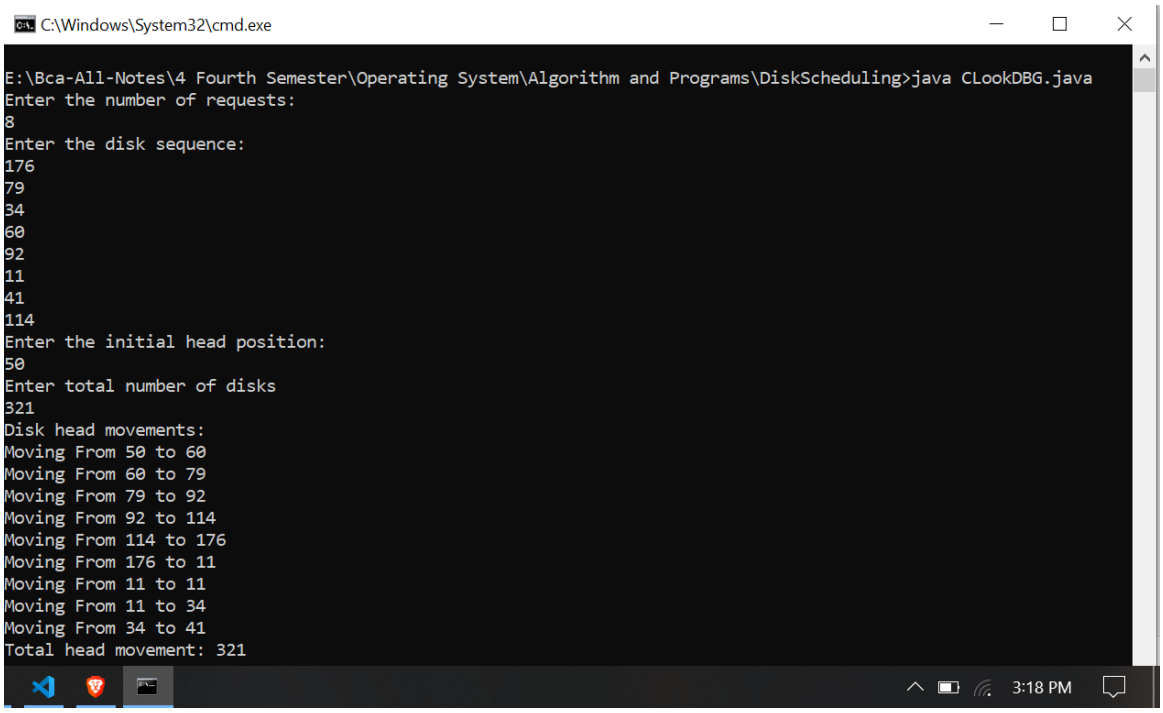
6. WAP to implement C-Look disk scheduling algorithm.

Source Code:

```
import java.util.Arrays;
import java.util.Scanner;
public class CLookDBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of requests:");
      int binti = sc.nextInt();
      int requests[] = new int[binti];
      System.out.println("Enter the disk sequence:");
      for (int b = 0; b < binti; b++) {
         requests[b] = sc.nextInt();
      }
      System.out.println("Enter the initial head position:");
      int suruPosition = sc.nextInt();
      System.out.println("Enter total number of disks");
      int totalDisks = sc.nextInt();
      Arrays.sort(requests);
      int totalHeadMovement = 0;
      int ailekoPosition = suruPosition;
      System.out.println("Disk head movements:");
      for (int j = 0; j < binti; j++) {
         if (requests[j] >= suruPosition) {
            System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
            ailekoPosition = requests[j];
         }
      }
```

```
        if (ailekoPosition > requests[0]) {
            System.out.println("Moving From " + ailekoPosition + " to " + requests[0]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[0]);
            ailekoPosition = requests[0];
        }
        for (int k = 0; k < binti; k++) {
            if (requests[k] < suruPosition) {
                System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
                ailekoPosition = requests[k];
            }
        }
        System.out.println("Total head movement: " + totalHeadMovement);
    }
}
```

Output:



Memory Allocation Algorithm

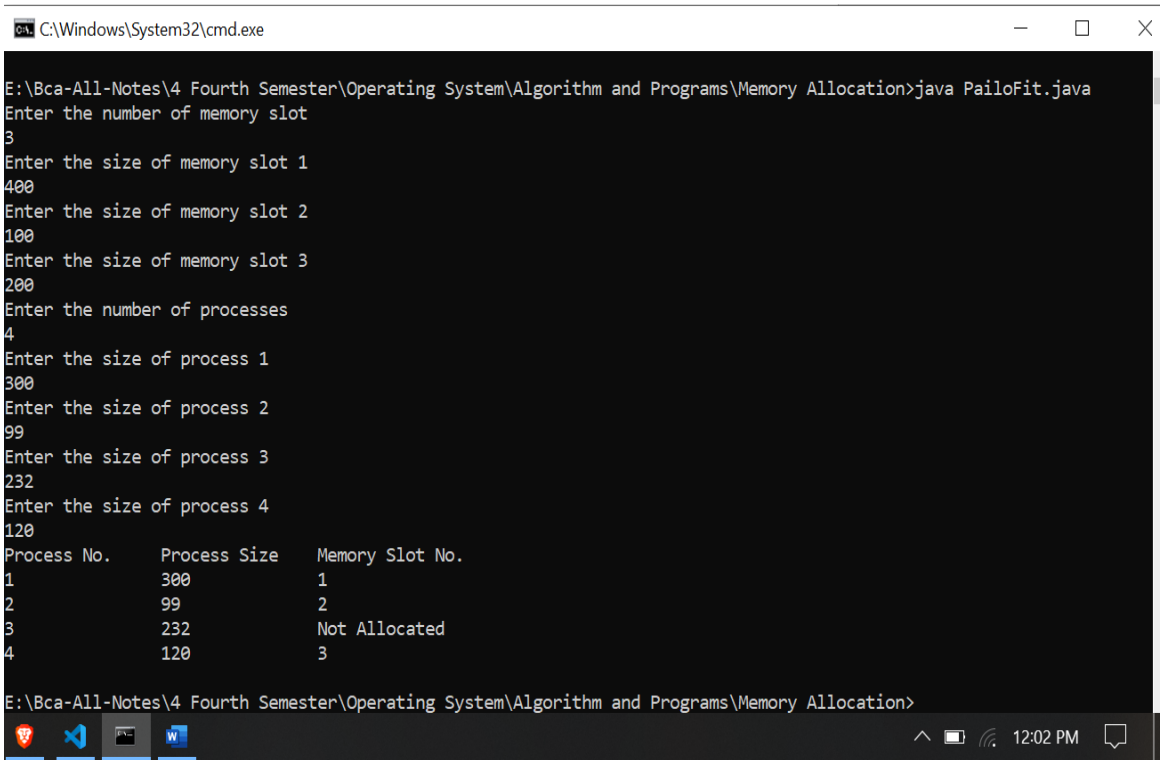
1. WAP to implement First Fit Memory Allocation Algorithm.

Source Code:

```
import java.util.Scanner;
public class PailoFit {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of memory slot");
      int slots = sc.nextInt();
      int[] memorySizes = new int[slots];
      boolean[] filled = new boolean[slots];
      for (int b = 0; b < slots; b++) {
         System.out.println("Enter the size of memory slot " + (b + 1));
         memorySizes[b] = sc.nextInt();
      }
      System.out.println("Enter the number of processes");
      int processes = sc.nextInt();
      int[] processSizes = new int[processes];
      int allocation[] = new int[processes];
      for (int b = 0; b < processes; b++) {
         System.out.println("Enter the size of process " + (b + 1));
         processSizes[b] = sc.nextInt();
         allocation[b] = -1;
      }
      for (int i = 0; i < processes; i++) {
         for (int j = 0; j < slots; j++) {
            if (!filled[j] && memorySizes[j] >= processSizes[i]) {
               allocation[i] = j;
               filled[j] = true;
               break;
```

```
            }
        }
    }
    System.out.println("Process No.\tProcess Size\tMemory Slot No.");
    for (int i = 0; i < processes; i++) {
        System.out.print((i + 1) + "\t\t" + processSizes[i] + "\t\t");
        if (allocation[i] != -1) {
            System.out.println((allocation[i] + 1));
        } else {
            System.out.println("Not Allocated");
        }
    }
    sc.close();
  }
}
```

Output:



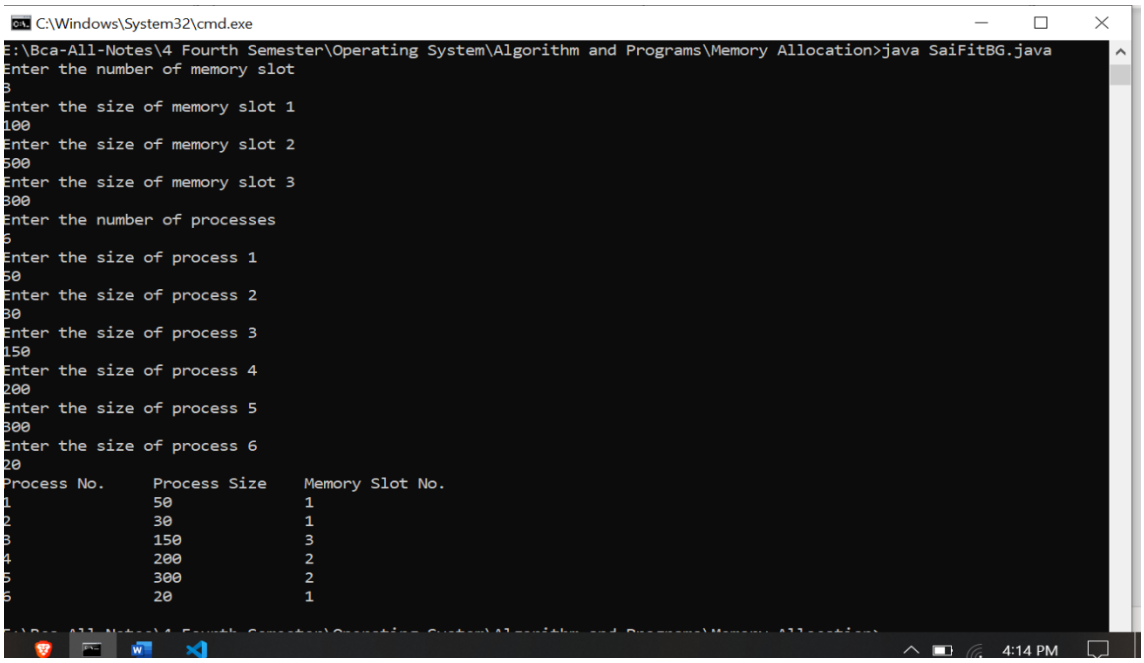
2. WAP to implement Best Fit Memory Allocation Algorithm.

Source Code:

```
import java.util.Scanner;
public class SaiFitBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of memory slot");
      int slots = sc.nextInt();
      int []memorySizes= new int [slots];
      boolean[]filled = new boolean[slots];
      for (int b=0;b<slots;b++){
         System.out.println("Enter the size of memory slot "+(b+1));
         memorySizes[b]=sc.nextInt();
     }
      System.out.println("Enter the number of processes");
      int processes = sc.nextInt();
      int []processSizes= new int [processes];
   int []allocation = new int[processes];
      for (int b=0;b<processes;b++){
         System.out.println("Enter the size of process "+(b+1));
         processSizes[b]=sc.nextInt();
         allocation[b]=-1;
      }
      for (int i=0;i<processes;i++){
         int bestIdx=-1;
         for (int j=0;j<slots;j++){
            if (!filled[j] && memorySizes[j]>=processSizes[i]){
               if (bestIdx==-1){
                  bestIdx=j;    }
               else if (memorySizes[j]<memorySizes[bestIdx]){
```

```
                    bestIdx=j;
                }}    }
            if (bestIdx!=-1){
                allocation[i]=bestIdx;
memorySizes[bestIdx] -= processSizes[i];}
        }
        System.out.println("Process No.\tProcess Size\tMemory Slot No.");
        for (int i=0;i<processes;i++){
            System.out.print((i+1)+"\t\t"+processSizes[i]+"\t\t");
            if (allocation[i]!=-1){
                System.out.println((allocation[i]+1));  }
            else {
                System.out.println("Not Allocated");
            } }
        sc.close();
    }
}
```

Output:



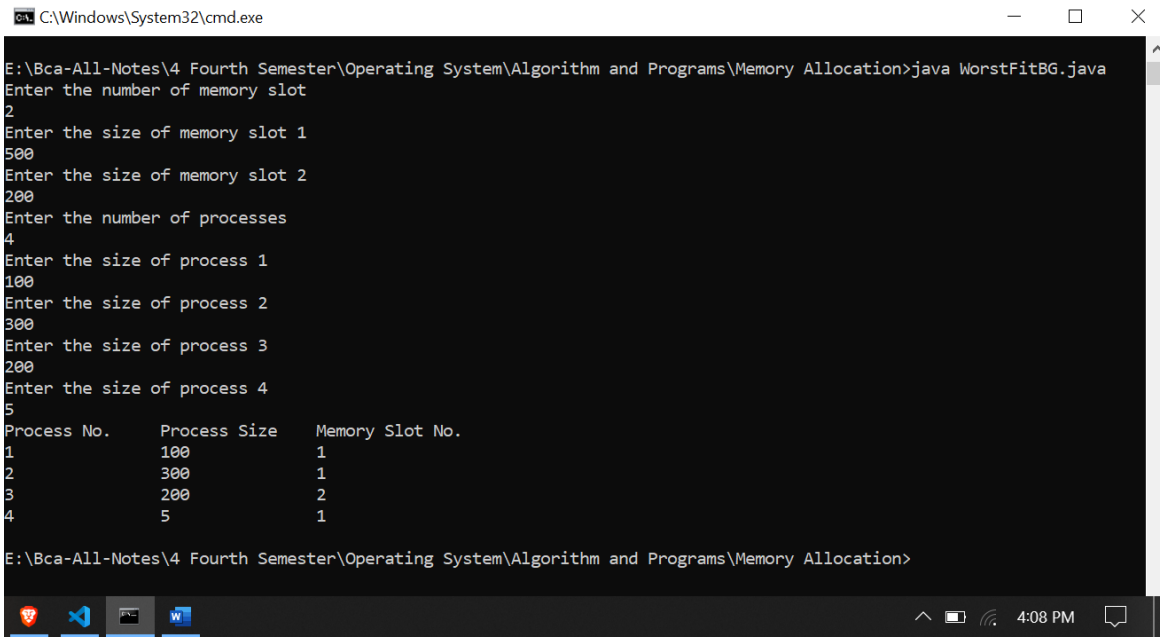
3. WAP to implement Worst Fit Memory Allocation Algorithm.

Source code:

```
import java.util.Scanner;
public class WorstFitBG{
   public static void main(String[] args){
            Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of memory slot");
      int slots = sc.nextInt();
      int []memorySizes= new int [slots];
      boolean[]filled = new boolean[slots];
      for (int b=0;b<slots;b++){
         System.out.println("Enter the size of memory slot "+(b+1));
         memorySizes[b]=sc.nextInt();
 }
      System.out.println("Enter the number of processes");
      int processes = sc.nextInt();
      int []processSizes= new int [processes];
   int []allocation = new int[processes];
      for (int b=0;b<processes;b++){
         System.out.println("Enter the size of process "+(b+1));
         processSizes[b]=sc.nextInt();
         allocation[b]=-1;    }
       for (int i = 0; i < processes; i++) {
         int worstIdx = -1;
         for (int j = 0; j < slots; j++) {
            if (!filled[j] && memorySizes[j] >= processSizes[i]) {
               if (worstIdx == -1 || memorySizes[j] > memorySizes[worstIdx]) {
                  worstIdx = j;
}
            }
```

```
}
            if (worstIdx != -1) {
                allocation[i] = worstIdx;
                memorySizes[worstIdx] -= processSizes[i];
            }
        }
            System.out.println("Process No.\tProcess Size\tMemory Slot No.");
        for (int i = 0; i < processes; i++) {
            System.out.print((i + 1) + "\t\t" + processSizes[i] + "\t\t");
            if (allocation[i] != -1) {
                System.out.println((allocation[i] + 1));
            } else {
                System.out.println("Not Allocated");
            }
        }
        sc.close();
    }
}
```

Output:



Deadlocks

1. WAP to implement Bankers Algorithm.

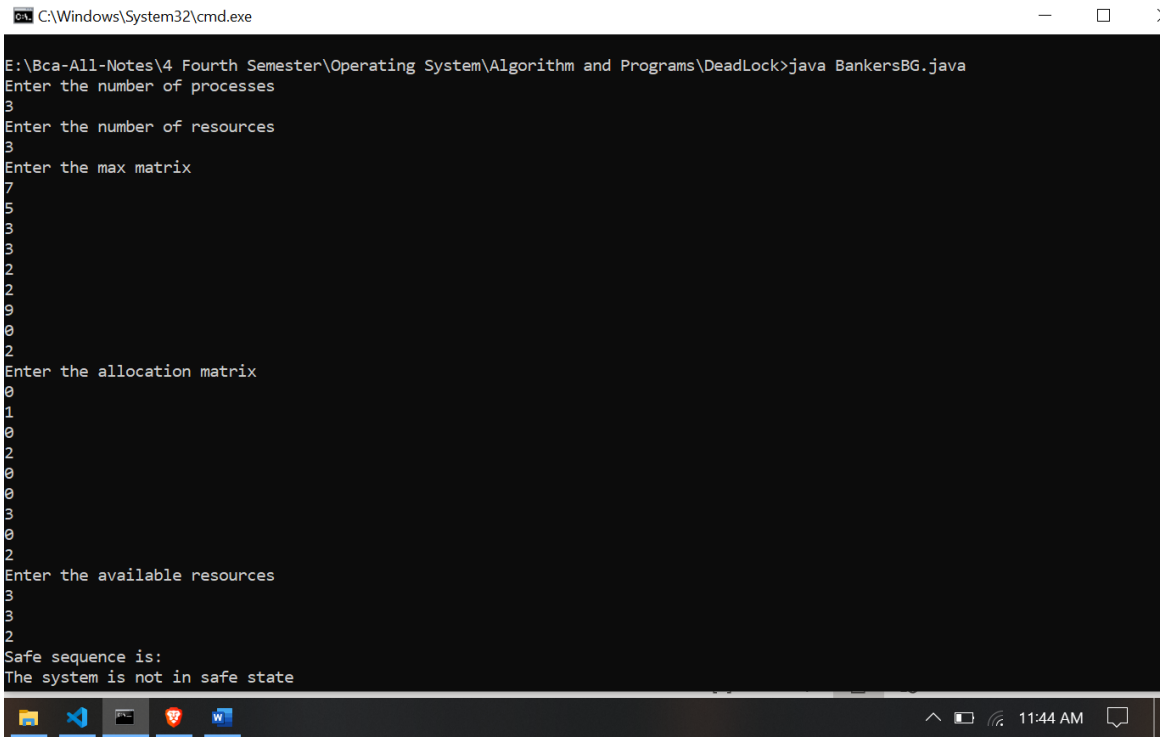
Source code:

```
import java.util.Scanner;


public class BankersBG {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of processes");
      int processNum = sc.nextInt();
      System.out.println("Enter the number of resources");
      int resourceNum = sc.nextInt();
      int[][] max = new int[processNum][resourceNum];
      int[][] allocation = new int[processNum][resourceNum];
      int[][] chiyeko = new int[processNum][resourceNum];
      int[] vako = new int[resourceNum];
      boolean[] visited = new boolean[processNum];
      int[] safe = new int[processNum];
      System.out.println("Enter the max matrix");
      for (int i = 0; i < processNum; i++) {
         for (int j = 0; j < resourceNum; j++) {
            max[i][j] = sc.nextInt();
         }
      }
      System.out.println("Enter the allocation matrix");
      for (int i = 0; i < processNum; i++) {
         for (int j = 0; j < resourceNum; j++) {
            allocation[i][j] = sc.nextInt();
            chiyeko[i][j] = max[i][j] - allocation[i][j];
         }
```

```
}
System.out.println("Enter the available resources");
for (int i = 0; i < resourceNum; i++) {
   vako[i] = sc.nextInt();
}
int c = 0;
System.out.println("Safe sequence is:");
while (c < processNum) {
   boolean vitiyo = false;
   for (int i = 0; i < processNum; i++) {
      if (visited[i] == false) {
         int j;
         for (j = 0; j < resourceNum; j++) {
            if (chiyeko[i][j] > vako[j]) {

               break;
            }
         }
         if (j == resourceNum) {
            for (int k = 0; k < resourceNum; k++) {
               vako[k] += allocation[i][k];
            }
            safe[c++] = i;
            visited[i] = true;
            vitiyo = true;
         }
      }
   }
   if (!vitiyo) {
      System.out.println("The system is not in safe state");
```

```
                return;
            }
        }
        System.out.print("Hence, the SAFE Sequence is as follows: ");
        for (int i = 0; i < processNum; i++) {
            System.out.print("P" + safe[i]);
            if (i != processNum - 1) {
                System.out.print(" -> ");
            }
        }
        System.out.println(".");
        sc.close();
    }
}
```

Output:



2. WAP to implement deadlock detection algorithm.

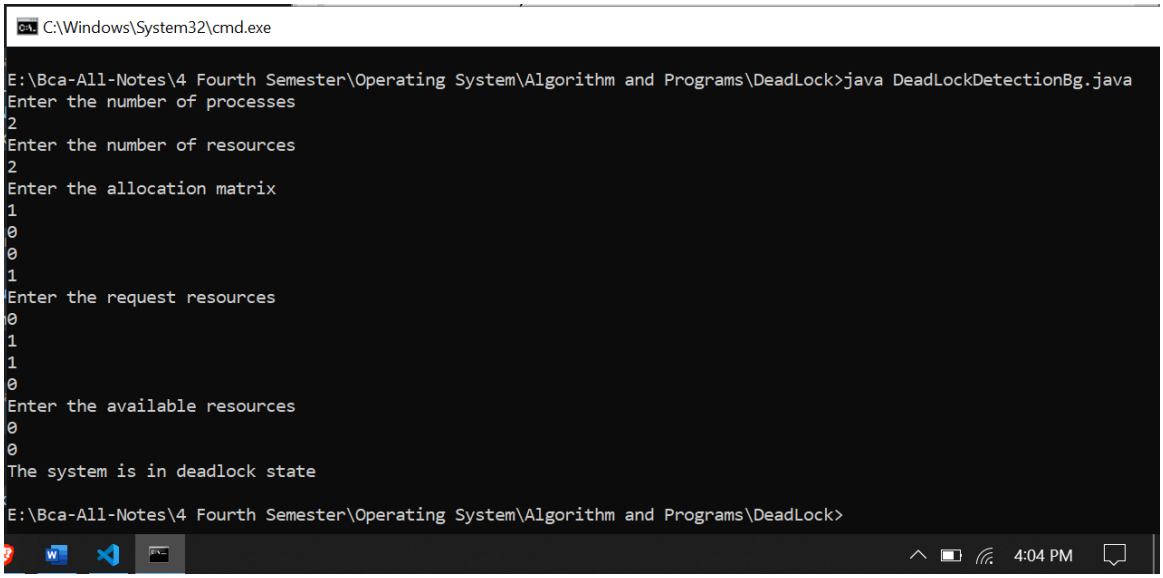
Code:

```
import java.util.Scanner;


public class DeadLockDetectionBg {
public static void main(String[] args){
   Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of processes");
      int processNum = sc.nextInt();
      System.out.println("Enter the number of resources");
      int resourceNum = sc.nextInt();



      int[][] allocation = new int[processNum][resourceNum];
      int[][] chiyeko = new int[processNum][resourceNum];
      int[] vako = new int[resourceNum];
      boolean[] visited = new boolean[processNum];



      System.out.println("Enter the allocation matrix");
      for (int i = 0; i < processNum; i++) {
         for (int j = 0; j < resourceNum; j++) {
            allocation[i][j] = sc.nextInt();


         }
      }
      System.out.println("Enter the request resources");
      for (int i = 0; i < resourceNum; i++) {
         for (int j = 0; j < processNum; j++) {
            chiyeko[i][j] = sc.nextInt();
         }
```

```
        }
        System.out.println("Enter the available resources");
        for (int i = 0; i < resourceNum; i++) {
            vako[i] = sc.nextInt();
        }
        if (isDeadLock(allocation, chiyeko, vako, visited, processNum, resourceNum)) {
            System.out.println("The system is in deadlock state");
        } else {
            System.out.println("The system is not in deadlock state");
        }
    }
    public static boolean isDeadLock(int[][] allocation, int[][] chiyeko, int[] vako, boolean[]
visited,
            int processNum, int resourceNum) {
        int c = 0;
        while (c < processNum) {
            boolean vitiyo = false;
            for (int i = 0; i < processNum; i++) {
                if (visited[i] == false) {
                    int j;
                    for (j = 0; j < resourceNum; j++) {
                        if (chiyeko[i][j] > vako[j]) {

                            break;
                        }
                    }
                    if (j == resourceNum) {
                        for (int k = 0; k < resourceNum; k++) {
                            vako[k] += allocation[i][k];
                        }
```

```
                    visited[i] = true;
                    vitiyo = true;
                    c++;
                }
            }
        }
        if (!vitiyo) {
            return true;
        }
    }
    return false;


  }
}
```

Output:



3. WAP to implement deadlock recovery.

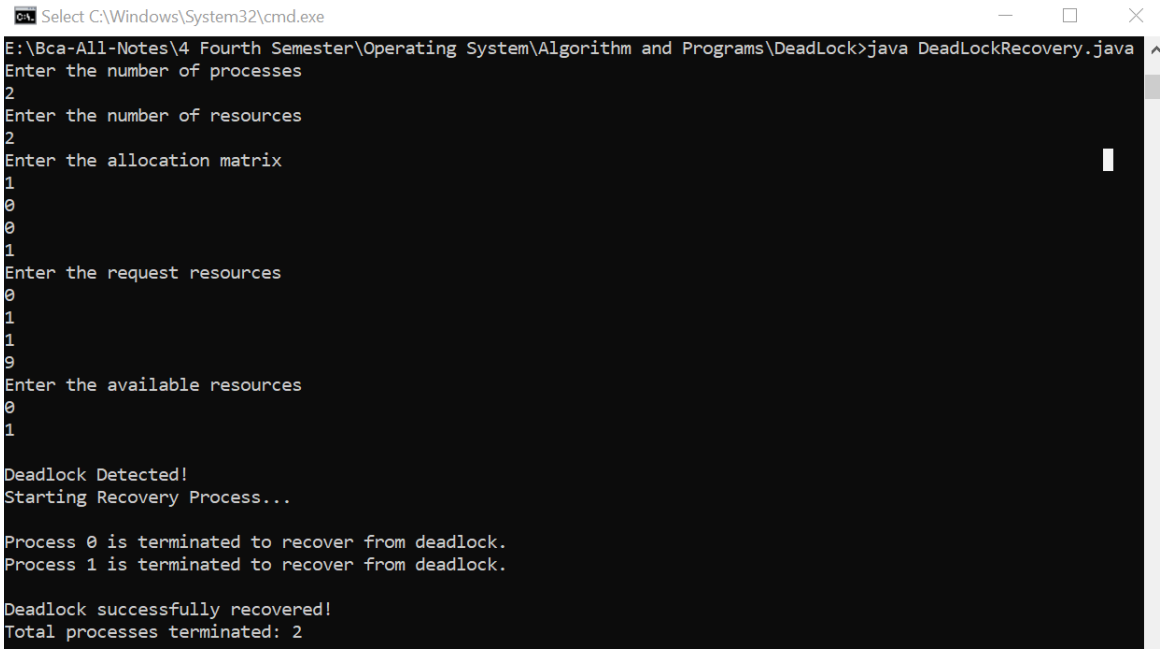
```
import java.util.Scanner;

public class DeadLockRecovery {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of processes");
      int processNum = sc.nextInt();
      System.out.println("Enter the number of resources");
      int resourceNum = sc.nextInt();

      int[][] allocation = new int[processNum][resourceNum];
      int[][] chiyeko = new int[processNum][resourceNum];
      int[] vako = new int[resourceNum];
      boolean[] visited = new boolean[processNum];

      System.out.println("Enter the allocation matrix");
      for (int i = 0; i < processNum; i++) {
         for (int j = 0; j < resourceNum; j++) {
            allocation[i][j] = sc.nextInt();
         }
      }
      System.out.println("Enter the request resources");
      for (int i = 0; i < processNum; i++) {
         for (int j = 0; j < resourceNum; j++) {
            chiyeko[i][j] = sc.nextInt();
         }
      }
      System.out.println("Enter the available resources");
      for (int i = 0; i < resourceNum; i++) {
```

```
            vako[i] = sc.nextInt();
        }

        boolean deadlockExists = isDeadlock(allocation, chiyeko, vako, visited,
processNum, resourceNum);

        if (deadlockExists) {
            System.out.println("\nDeadlock Detected!");
            System.out.println("Starting Recovery Process...\n");
            boolean[] visited2 = new boolean[processNum];
            deadlockrecovery(allocation, chiyeko, vako, visited2, processNum,
resourceNum);
        } else {
            System.out.println("\nNo Deadlock.  System is Safe.");
        }
    }

    public static boolean isDeadlock(int[][] allocation, int[][] chiyeko, int[] vako, boolean[]
visited,
            int processNum, int resourceNum) {
        int c = 0;
        int[] vakoTemp = new int[resourceNum];
        for (int i = 0; i < resourceNum; i++) {
            vakoTemp[i] = vako[i];
        }

        while (c < processNum) {
            boolean vitiyo = false;
            for (int i = 0; i < processNum; i++) {
                if (visited[i] == false) {
                    int j;
```

```
                for (j = 0; j < resourceNum; j++) {
                    if (chiyeko[i][j] > vakoTemp[j]) {
                        break;
                    }
                }
                if (j == resourceNum) {
                    for (int k = 0; k < resourceNum; k++) {
                        vakoTemp[k] += allocation[i][k];
                    }
                    visited[i] = true;
                    vitiyo = true;
                    c++;
                }
            }
        }
        if (!vitiyo) {
            return true;
        }
    }
    return false;
}

public static void deadlockrecovery(int[][] allocation, int[][] chiyeko, int[] vako,
boolean[] visited,
        int processNum, int resourceNum) {
    int c = 0;
    int terminatedCount = 0;

    while (c < processNum) {
        boolean vitiyo = false;
        for (int i = 0; i < processNum; i++) {
```

```
            if (visited[i] == false) {
                int j;
                for (j = 0; j < resourceNum; j++) {
                    if (chiyeko[i][j] > vako[j]) {
                        break;
                    }
                }
                if (j == resourceNum) {
                    for (int k = 0; k < resourceNum; k++) {
                        vako[k] += allocation[i][k];
                    }
                    visited[i] = true;
                    vitiyo = true;
                    c++;
                    terminatedCount++;
                    System.out.println("Process " + i + " is terminated to recover from
deadlock.");
                }
            }
        }
        if (!vitiyo) {
            for (int i = 0; i < processNum; i++) {
                if (visited[i] == false) {
                    System.out.println("Process " + i + " is terminated to recover from
deadlock.");
                    for (int k = 0; k < resourceNum; k++) {
                        vako[k] += allocation[i][k];
                    }
                    visited[i] = true;
                    terminatedCount++;
                    c++;
```

```
                }
            }
            if (c < processNum) {
                System.out.println("\nDeadlock cannot be resolved further.");
            } else {
                System.out.println("\nDeadlock successfully recovered!");
                System.out.println("Total processes terminated: " + terminatedCount);
            }
            return;
        }
    }
    System.out.println("\nDeadlock successfully recovered!");
    System.out.println("Total processes terminated: " + terminatedCount);
  }
}
```



Classical IPC Problems

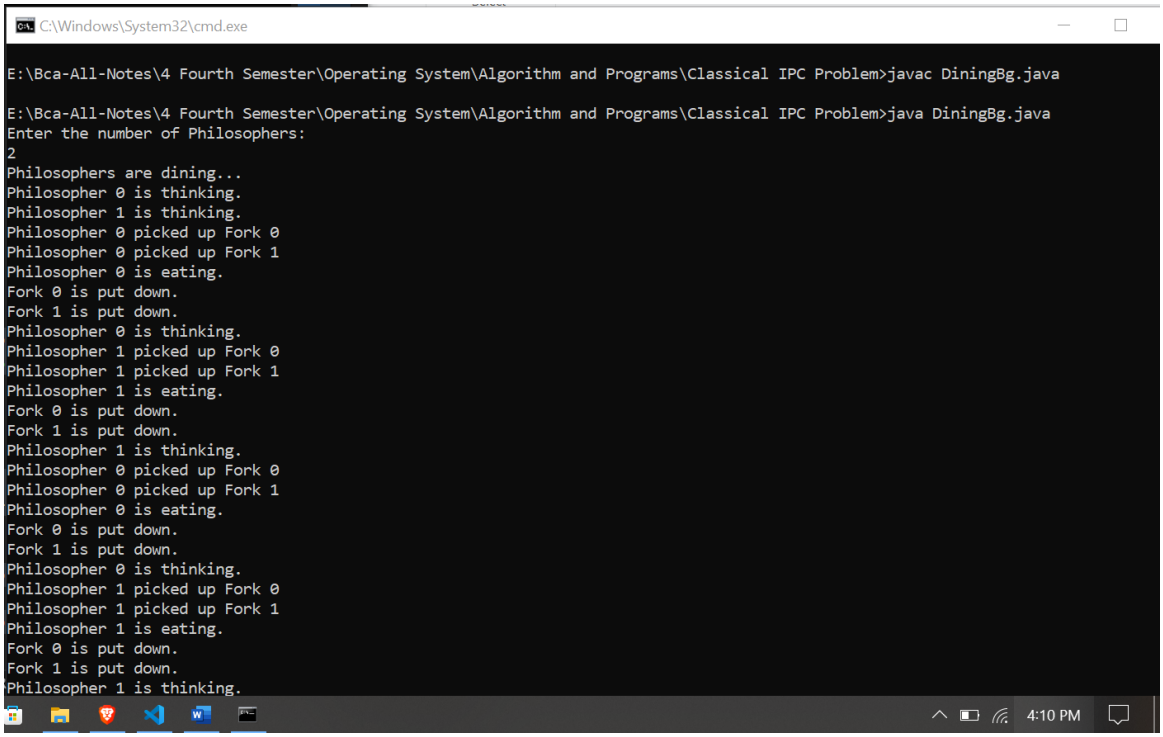
1. WAP to implement Dining Philosopher Problem Algorithm.

Code:

```
import java.util.Scanner;
public class DiningBg {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of Philosophers:");
      int n = sc.nextInt();
      Philosopher[] philosophers = new Philosopher[n];
      Fork[] forks = new Fork[n];
      for (int i = 0; i < n; i++) {
         forks[i] = new Fork(i);
      }
      for (int i = 0; i < n; i++) {
         philosophers[i] = new Philosopher(i, forks[i], forks[(i + 1) % n]);
         new Thread(philosophers[i]).start();
      }
      System.out.println("Philosophers are dining...");
      sc.close();
   }
}
class Fork {
   public final int id;
   private boolean prayog;
   public Fork(int id) {
      this.id = id;
      this.prayog = false;
   }
   public synchronized void pickUp(int philosopherId) throws InterruptedException {
      while (prayog) {
```

```
            wait();
        }
        prayog = true;
        System.out.println("Philosopher " + philosopherId + " picked up Fork " + id);
    }
    public synchronized void putDown() {
        prayog = false;
        System.out.println("Fork " + id + " is put down.");
        notifyAll();
    }
}
class Philosopher implements Runnable {
    private final int id;
    private final Fork dabareFork;
    private final Fork rightFork;
    public Philosopher(int id, Fork dabareFork, Fork rightFork) {
        this.id = id;
        this.dabareFork = dabareFork;
        this.rightFork = rightFork;
    }
    @Override
    public void run() {
        try {
            while (true) {
                think();
                eat();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
```

```
    }
    private void think() throws InterruptedException {
        System.out.println("Philosopher " + id + " is thinking.");
        Thread.sleep((long) (Math.random() * 1000));
    }
    private void eat() throws InterruptedException {
        Fork first = dabareFork.id < rightFork.id ? dabareFork : rightFork;
        Fork second = dabareFork.id < rightFork.id ? rightFork : dabareFork;
        first.pickUp(id);
        second.pickUp(id);
        System.out.println("Philosopher " + id + " is eating.");
        Thread.sleep((long) (Math.random() * 1000));
        first.putDown();
        second.putDown();
    }
}
```

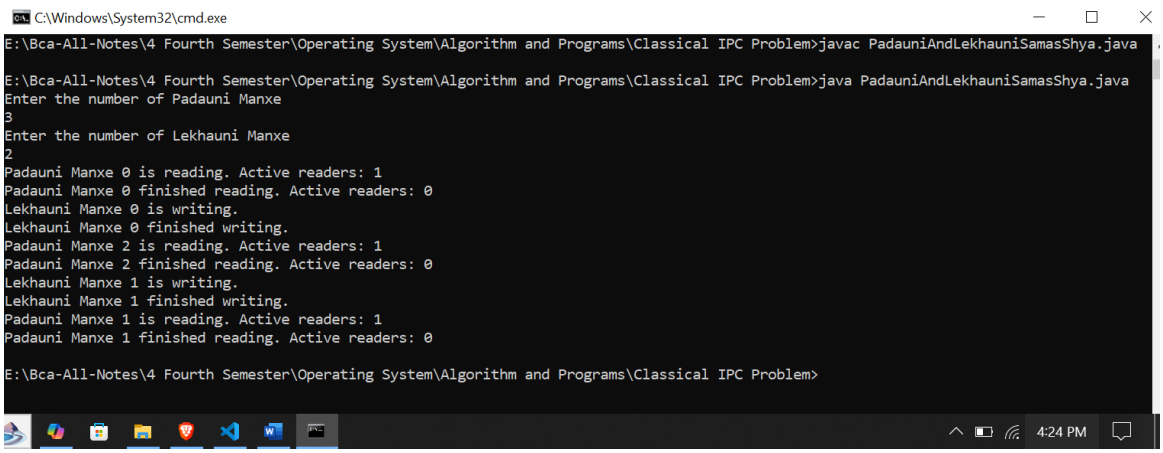


2. WAP to implement Readers and Writes Problem Algorithm.

Code:

```
import java.util.Scanner;
public class PadauniAndLekhauniSamasShya {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of Padauni Manxe");
      int padauni = sc.nextInt();
      System.out.println("Enter the number of Lekhauni Manxe");
      int lekhauni = sc.nextInt();
      BadekoData badekoData = new BadekoData(padauni, lekhauni);
      for (int i = 0; i < padauni; i++) {
         int padauniId = i;
         new Thread() {public void run() {
               try {                    badekoData.padauniKaam(padauniId);
               } catch (InterruptedException e) {
                  e.printStackTrace();                    }    }.start();  }
      for (int i = 0; i < lekhauni; i++) {
         int lekhauniId = i;
         new Thread() {      public void run() {  try {
                  badekoData.lekhauniKaam(lekhauniId);
               } catch (InterruptedException e) {
                  e.printStackTrace();    }  }}.start();}
      sc.close();  }}
class BadekoData {
   private int padauni;  private int lekhauni;
   private int padauniCount = 0;
   private boolean lekhauniActive = false;

   public BadekoData(int padauni, int lekhauni) {
      this.padauni = padauni;
```

```
        this.lekhauni = lekhauni;
    }
    public synchronized void padauniKaam(int id) throws InterruptedException {
        while (lekhauniActive) {
            wait();  }
        padauniCount++;
        System.out.println("Padauni Manxe " + id + " is reading. Active readers: " +
padauniCount);
        Thread.sleep((long) (Math.random() * 1000));
        padauniCount--;
        System.out.println("Padauni Manxe " + id + " finished reading. Active readers: " +
padauniCount);
        if (padauniCount == 0) {
            notifyAll();}     }
    public synchronized void lekhauniKaam(int id) throws InterruptedException {
        while (lekhauniActive || padauniCount > 0) {  wait();}
        lekhauniActive = true;
        System.out.println("Lekhauni Manxe " + id + " is writing.");
        Thread.sleep((long) (Math.random() * 1000));
        lekhauniActive = false;
        System.out.println("Lekhauni Manxe " + id + " finished writing.");
        notifyAll();   }
}
```

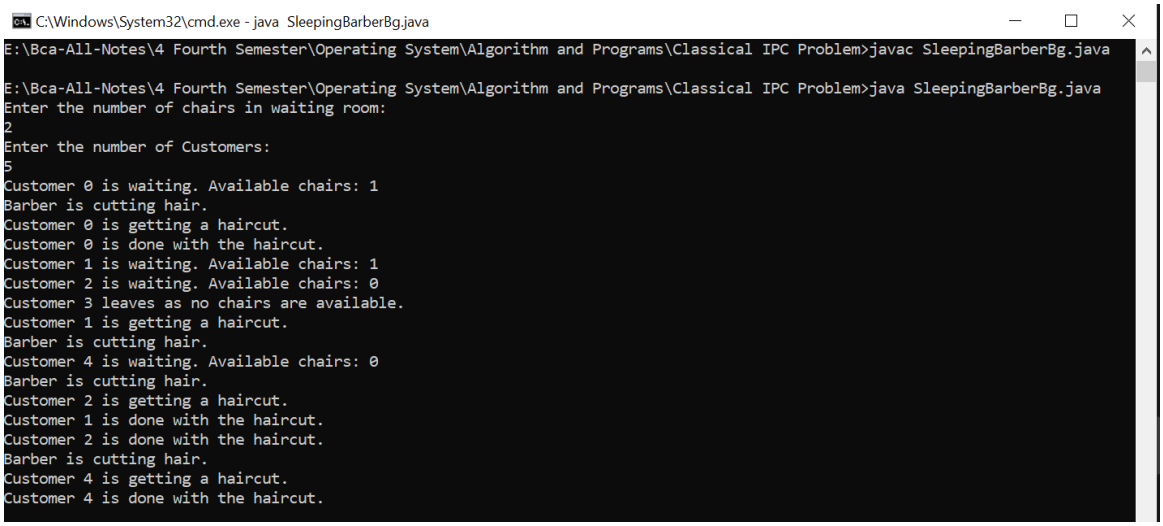


3. WAP to implement Sleep Barbers problem algorithm.

Code:

```
import java.util.Scanner;
import java.util.concurrent.Semaphore;
public class SleepingBarberBg {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter the number of chairs in waiting room:");
      int waitingChairs = sc.nextInt();
      System.out.println("Enter the number of Customers:");
      int customerCount = sc.nextInt();
      BarberShop barberShop = new BarberShop(waitingChairs);
      new Thread(() -> barberShop.barberKam()).start();
      for (int i = 0; i < customerCount; i++) {    int customerId = i;
         new Thread(() -> barberShop.customerArrivers(customerId)).start();
         try {           Thread.sleep((long) (Math.random() * 500));
         } catch (InterruptedException e) {              e.printStackTrace();
         }}  }}
class BarberShop {
   private final Semaphore barberReady;
   private final Semaphore customerReady;
   private final Semaphore accessSeats;
   private int khaliChairs;
   public BarberShop(int khurchi) {
      this.barberReady = new Semaphore(0);
      this.customerReady = new Semaphore(0);
      this.accessSeats = new Semaphore(1);
      this.khaliChairs = khurchi;    }
   public void barberKam() {
      while (true) {    try {
            customerReady.acquire();
```

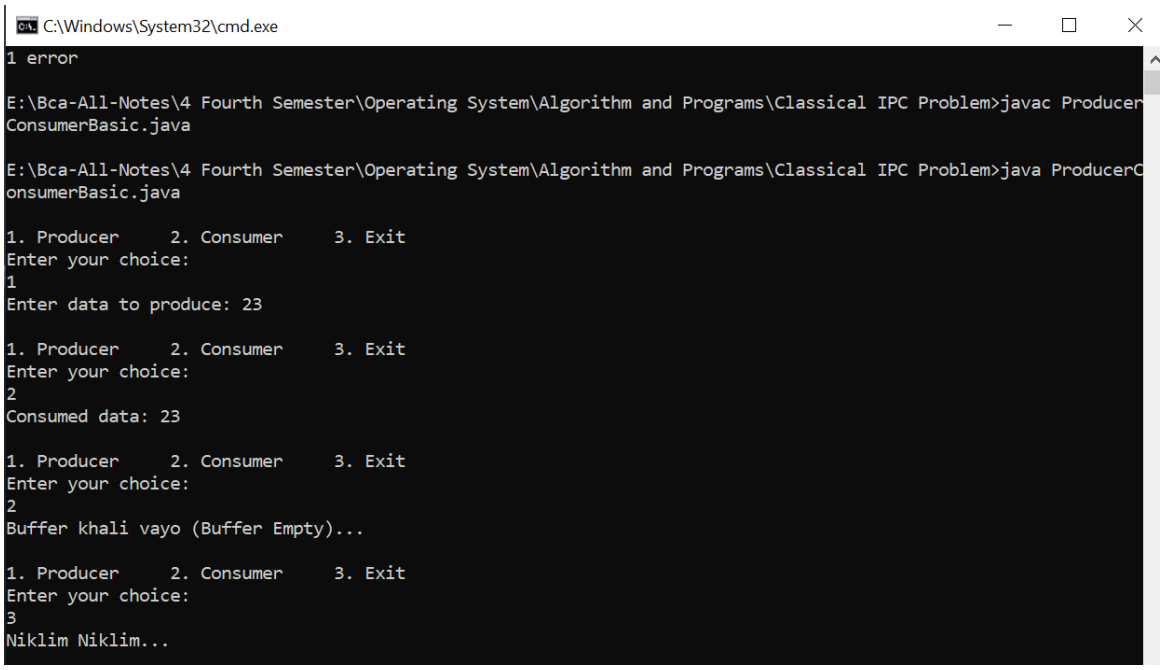
```
            accessSeats.acquire();   khaliChairs++;
            barberReady.release();     accessSeats.release();
            System.out.println("Barber is cutting hair.");
            Thread.sleep((long) (Math.random() * 1000));
        } catch (InterruptedException e) {
            e.printStackTrace();  }  }  }
   public void customerArrivers(int id) {
      try {  accessSeats.acquire();
         if (khaliChairs > 0) {  khaliChairs--;
            System.out.println("Customer " + id + " is waiting. Available chairs: " +
khaliChairs);
            customerReady.release();     accessSeats.release();      barberReady.acquire();
            System.out.println("Customer " + id + " is getting a haircut.");
            Thread.sleep((long) (Math.random() * 1000));
            System.out.println("Customer " + id + " is done with the haircut.");
   } else {  accessSeats.release();
            System.out.println("Customer " + id + " leaves as no chairs are
available.");       }
      } catch (InterruptedException e) {
e.printStackTrace();}
   }
}
```



1. WAP to implement producer-consumer problem.

```
import java.util.Scanner;
public class ProducerConsumerBasic {
public static void main(String[] args) {
int buffer_Aakar = 5;
int[] buffer = new int[buffer_Aakar];
int vitra = 0, bira = 0;
int choice = 0;
Scanner hehe = new Scanner(System.in);
while (choice != 3) {
System.out.println("\n1. Producer\t2. Consumer\t3. Exit\nEnter your choice:");
choice = hehe.nextInt();
switch (choice) {
case 1:
if ((vitra + 1) % buffer_Aakar == bira) {
System.out.println("Buffer Voriyo (Buffer Full)...");
} else {
System.out.print("Enter data to produce: ");
buffer[vitra] = hehe.nextInt();
vitra = (vitra + 1) % buffer_Aakar;
}
break;
case 2:
if (bira == vitra) {
System.out.println("Buffer khali vayo (Buffer Empty)...");
} else {
int consumedData = buffer[bira];
System.out.println("Consumed data: " + consumedData);
bira = (bira + 1) % buffer_Aakar;
}
break;
```

```
case 3:
System.out.println("Niklim Niklim...");
break;
default:
System.out.println("Invalid choice, try again.");
}
}
hehe.close();
}
}
```



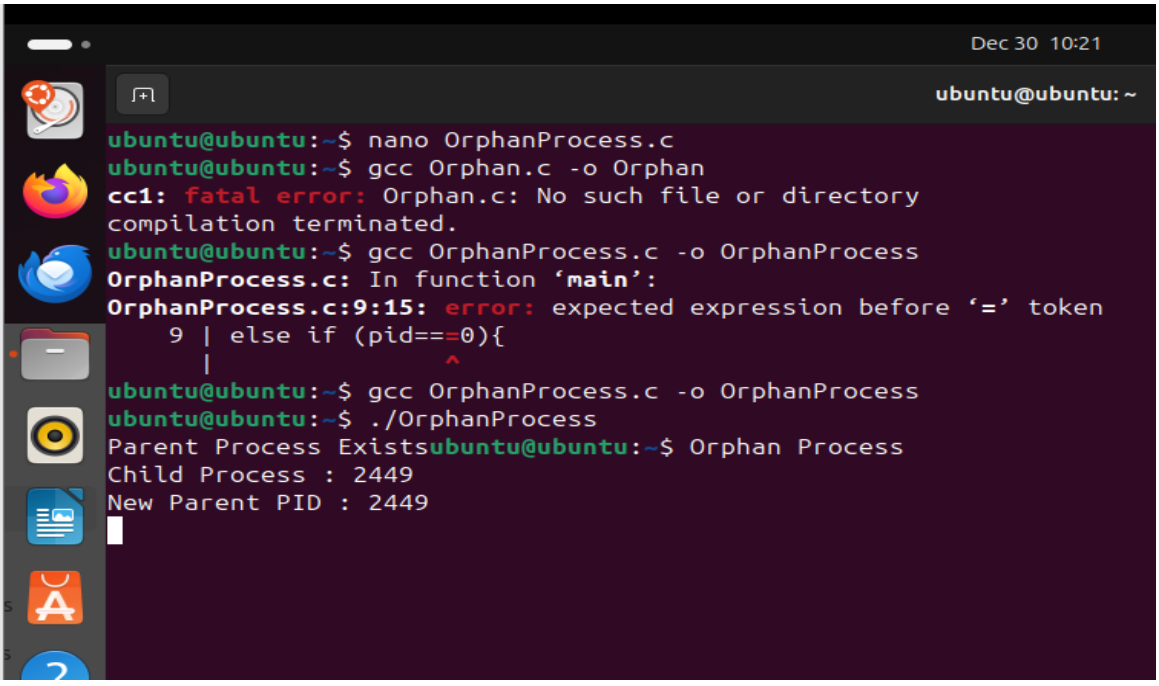
Process Creation

1. WAP to create Parent or child process.

```
#include <stdio.h>

#include <unistd.h>

int main() {

   pid_t pid = fork();

   if (pid == 0) {

      // Child process

      printf("Child Process\n");

      printf("Child PID: %d\n", getpid());

      printf("Parent PID: %d\n", getppid());

   }

   else if (pid > 0) {

      // Parent process

      printf("Parent Process\n");

      printf("Parent PID: %d\n", getpid());

   }

   else {

      printf("Fork failed\n");

   }

   return 0;

}
```

2. WAP to create orphan process.

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid > 0) {
        // Parent process exits early
        printf("Parent Process Exists\n");
        return 0;
    }
    else if (pid == 0) {
        sleep(5);
        printf("Orphan Process\n");
        printf("Child PID: %d\n", getpid());
        printf("New Parent PID (init/systemd): %d\n", getppid());
    }
    return 0;
}
```



3. WAP to create a demon Process

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        exit(EXIT_FAILURE);
    }
    if (pid > 0) {
        exit(EXIT_SUCCESS);
    }
    if (setsid() < 0) {
        exit(EXIT_FAILURE);
    }
    chdir("/");
    close(STDIN_FILENO);
    close(STDOUT_FILENO);
    close(STDERR_FILENO);
    while (1) {
        sleep(10);
    }
    return 0;
}
```